

Comprendre le projet ligne par ligne

Bibliothèque Universitaire — chaque fichier qui prouve la maîtrise des trois points annoncés par le professeur : Inversion de Contrôle / Injection de Dépendances, JPA, et API REST sécurisée par token JWT.

IoC + DI

JPA / Hibernate

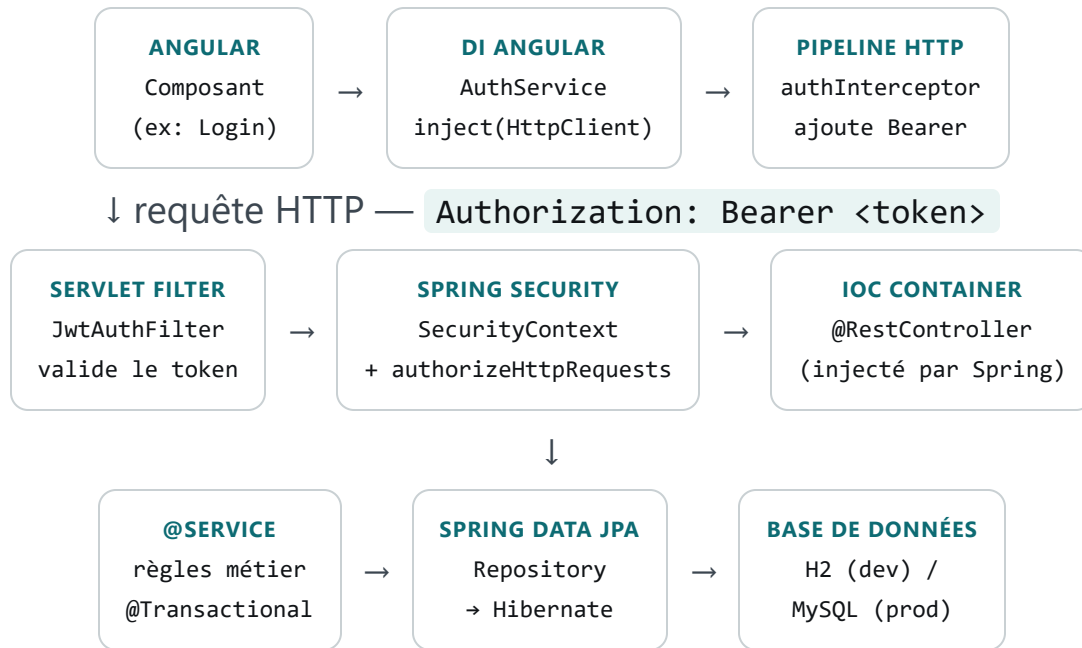
REST + JWT

Angular (client)

Mode d'emploi. Les fichiers qui portent les trois notions attendues sont expliqués intégralement, ligne par ligne. Les fichiers répétitifs (accesseurs get/set, autres contrôleurs CRUD identiques) sont montrés en entier pour que vous puissiez citer n'importe quelle ligne, mais commentés une seule fois. Le code affiché est **fidèle au fichier réel** — la numérotation des lignes correspond exactement à `chemin/fichier.java:ligne`, pratique pour répondre en pointant l'écran.

Vue d'ensemble : le trajet d'une requête

Avant le détail fichier par fichier, voici comment les trois notions s'articulent dans une seule requête authentifiée — c'est la réponse à donner si le professeur demande « *expliquez l'architecture globale* » avant d'entrer dans le code.



Le contrôleur ne renvoie jamais l'entité JPA : un DtoMapper la convertit en DTO (record) avant sérialisation JSON.

Trois couches d'IoC sont à l'œuvre en même temps : le conteneur Spring (backend) instancie et câble Controller → Service → Repository sans qu'aucune ligne de code ne fasse `new XxxService()` ; côté client, l'injecteur Angular fait la même chose pour AuthService et HttpClient via `inject()`.

Ce que chaque starter Maven active automatiquement

C'est la première brique de l'IoC côté Spring Boot : ajouter une dépendance déclenche une **auto-configuration** qui enregistre elle-même des beans dans le conteneur, sans configuration XML ni `new` manuel.

DÉPENDANCE (POM.XML)	CE QU'ELLE AUTO-CONFIGURE
<code>spring-boot-starter-webmvc</code>	DispatcherServlet, conversion JSON, résolution des <code>@RestController</code> — le cœur de l'API REST.
<code>spring-boot-starter-data-jpa</code>	EntityManagerFactory, DataSource, proxies d'implémentation des interfaces <code>JpaRepository</code> .
<code>spring-boot-starter-security</code>	La chaîne de filtres servlet (dont on personnalise une partie dans <code>SecurityConfig</code>), le contexte de sécurité.
<code>spring-boot-starter-validation</code>	Le moteur Bean Validation qui interprète <code>@NotBlank</code> , <code>@Email</code> , <code>@Size</code> sur les DTOs via <code>@Valid</code> .
<code>io.jsonwebtoken:jjwt-*</code>	La bibliothèque JWT utilisée par <code>JwtService</code> pour signer/vérifier les tokens (pas fournie par Spring).
<code>h2 / mysql-connector-j</code>	Pilotes JDBC — H2 en profil <code>dev</code> , MySQL en profil par défaut (prod du TP).

QUESTION PROBABLE DU JURY

« Où est configuré Spring MVC / Spring Security ? Vous n'avez pas de fichier XML... » — Réponse : Spring Boot n'utilise pas de XML. La présence d'une dépendance sur le classpath (`pom.xml`) suffit à déclencher son *auto-configuration* ; on ne surcharge que ce qu'on veut personnaliser (ex: `SecurityConfig`), le reste reste implicite.

1 IoC & Injection de Dépendances

Qui crée les objets, et qui les relie entre eux ?

L'inversion de contrôle (IoC) : au lieu que votre code fasse `new LivreService(new LivreRepository(...))`, c'est le **conteneur Spring** qui construit chaque objet (« bean ») et lui fournit ses dépendances tout seul — c'est l'**injection de dépendances** (DI), la mise en œuvre concrète de l'IoC. Dans tout ce projet, la technique utilisée est l'**injection par constructeur** : chaque classe déclare un seul constructeur listant ce dont elle a besoin, et Spring le remplit automatiquement.

BuBackend/.../BibliothequeApplication.java

POINT D'ENTRÉE

Le fichier le plus court, mais celui qui démarre tout le conteneur IoC.

```
1. package com.projet.bibliotheque;
2.
3. import org.springframework.boot.SpringApplication;
4. import org.springframework.boot.autoconfigure.SpringBootApplication;
5. import org.springframework.scheduling.annotation.EnableScheduling;
6.
7. @SpringBootApplication
8. @EnableScheduling
9. public class BibliothequeApplication {
10.
11.     public static void main(String[] args) {
12.
13.         SpringApplication.run(BibliothequeApplication.class, args);
14.
15.     }
16.
17. }
```

L.7

`@SpringBootApplication` = raccourci pour trois annotations : `@Configuration` (cette classe peut déclarer des beans), `@EnableAutoConfiguration` (déclenche l'auto-config vue plus haut) et `@ComponentScan` (Spring parcourt le package `com.projet.bibliotheque` et tous ses sous-packages à la recherche de `@Component`, `@Service`, `@Repository`, `@RestController` ... pour les enregistrer comme beans). C'est le point de départ de l'IoC : sans ce scan, aucune classe ne serait détectée.

L.8

`@EnableScheduling` active le support des méthodes `@Scheduled` (utilisées dans `SchedulerService` pour les tâches automatiques : retards, expirations de réservation). Sans elle, ces méthodes ne seraient jamais déclenchées.

L.11-13

`SpringApplication.run(...)` démarre le conteneur : il crée l' `ApplicationContext` (le registre de tous les beans), exécute le scan, instancie chaque bean dans le bon ordre de dépendances, puis lance le serveur web embarqué (Tomcat) et enfin tous les `CommandLineRunner` (dont `DataInitializer`, voir 1.4).

Un exemple pur d'IoC « explicite » : au lieu de laisser Spring instancier une classe annotée, on lui dit nous-mêmes comment fabriquer certains beans via des méthodes `@Bean`.

```
1. package com.projet.bibliotheque.config;
2.
3. import org.springframework.context.annotation.Bean;
4. import org.springframework.context.annotation.Configuration;
5. import org.springframework.security.authentication.AuthenticationManager;
6. import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
7. import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
8. import org.springframework.security.crypto.password.PasswordEncoder;
9.
10. /**
11.  * Beans transverses de sécurité, isolés de {@link SecurityConfig} pour éviter un
12.  * cycle de dépendances (SecurityConfig dépend de UserDetailsService, lui-même
13.  * dépendant du PasswordEncoder).
14.  */
15. @Configuration
16. public class BeansConfig {
17.
18.     @Bean
19.     public PasswordEncoder passwordEncoder() {
20.         return new BCryptPasswordEncoder();
21.     }
22.
23.     @Bean
```

```
4.     public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {  
5.  
6.         return config.getAuthenticationManager();  
7.  
8.     }  
9.  
10. }
```

L.10-13

Le commentaire donne la vraie raison d'être du fichier : c'est un piège classique de DI. `SecurityConfig` a besoin d'un `UserDetailsService` (→ `UserService`) qui a besoin d'un `PasswordEncoder` ; si le `PasswordEncoder` était déclaré directement dans `SecurityConfig`, Spring pourrait entrer dans un ordre d'initialisation ambigu. Le sortir dans une classe `@Configuration` séparée casse le cycle.

L.15

`@Configuration` indique à Spring que cette classe contient des **définitions de beans** (les méthodes `@Bean`). C'est une spécialisation de `@Component` : la classe elle-même est donc aussi scannée et enregistrée.

L.18-21

`passwordEncoder()` : chaque appel de méthode annotée `@Bean` enregistre son objet retourné dans le conteneur, indexé par le type de retour (`PasswordEncoder`) et le nom de la méthode. N'importe quelle autre classe peut ensuite demander un `PasswordEncoder` dans son constructeur — Spring injectera cette même instance (portée par défaut : **singleton**).

L.23-26

`authenticationManager(AuthenticationConfiguration config)` montre qu'une méthode `@Bean` peut elle-même recevoir des paramètres injectés — ici `AuthenticationConfiguration` est un bean fourni par Spring Security. On l'utilise pour récupérer l' `AuthenticationManager` par défaut (celui qui orchestrera `UserDetailsService` + `PasswordEncoder` lors du login, voir 3.5).

QUESTION PROBABLE DU JURY

« Quelle est la différence entre `@Component` et `@Bean` ? » — `@Component` (et ses spécialisations `@Service` / `@Repository` / `@RestController`) s'applique **sur la classe elle-même** : c'est Spring qui l'instancie en trouvant un constructeur. `@Bean` s'applique **sur une méthode** à l'intérieur d'une classe `@Configuration` : utile quand on ne possède pas la classe (ex: `BCryptPasswordEncoder` vient d'une bibliothèque tierce, on ne peut pas ajouter `@Component` dessus) ou quand la construction nécessite de la logique.

BuBackend/.../config/SecurityConfig.java

EXTRAIT — ANGLE IOC

Le détail complet de ce fichier (règles d'autorisation, filtres, CORS) est traité en [3.3](#) — spécification REST/JWT oblige. Ici, on regarde uniquement **comment il est câblé** par le conteneur.

```
8. public class SecurityConfig {
9.
10.     private final JwtService jwtService;
11.
12.     private final UserDetailsService userDetailsService;
13.
14.     private final JsonMapper jsonMapper;
15.
16.     public SecurityConfig(JwtService jwtService, UserDetailsService userDetailsService, JsonMapper jsonMapper) {
17.
18.         this.jwtService = jwtService;
19.
20.         this.userDetailsService = userDetailsService;
21.
22.         this.jsonMapper = jsonMapper;
23.
24.     }
25.
26.     @Bean
27.
28.     public JwtAuthFilter jwtAuthFilter() {
29.
30.         return new JwtAuthFilter(jwtService, userDetailsService);
31.
32.     }
33. }
```

L.34-38

Constructeur unique avec 3 dépendances : depuis Spring 4.3, quand une classe n'a **qu'un seul constructeur**, l'annotation `@Autowired` est implicite — inutile de l'écrire. Spring résout chaque paramètre par **type** :

`JwtService` (bean `@Component` , voir 3.1), `UserDetailsService` (implémenté par `UserService` , voir 3.6) et `JsonMapper` (bean Jackson 3 auto-configuré par le starter web).

L.40-43

Point subtil et excellent exemple de maîtrise IoC : `jwtAuthFilter()` fabrique le filtre **manuellement avec new** (car `JwtAuthFilter` n'est pas une classe `@Component` — il a besoin d'arguments de constructeur qui

ne sont disponibles qu'ici). Mais comme la méthode est annotée `@Bean` à l'intérieur d'une classe `@Configuration`, Spring l'enregistre quand même dans le conteneur comme un bean à part entière.

PIÈGE À CONNAÎTRE — MODE « FULL » DES @CONFIGURATION

Plus loin dans le fichier, `filterChain()` appelle `jwtAuthFilter()` directement (comme un appel de méthode Java classique) plutôt que de le recevoir en paramètre. Normalement, appeler deux fois une méthode `@Bean` devrait construire deux objets différents. Ici, non : Spring génère au démarrage une **sous-classe CGLIB** de `SecurityConfig` qui intercepte les appels internes à des méthodes `@Bean` et renvoie **l'instance déjà enregistrée** dans le conteneur au lieu de ré-exécuter le corps de la méthode. C'est le mode « full configuration » (celui par défaut ; l'alternative serait `@Configuration(proxyBeanMethods = false)`, plus rapide mais qui perdrait cette garantie de singleton). Si le jury demande « donc `new JwtAuthFilter()` est appelé deux fois ? » → non, justement grâce au proxy CGLIB.

BuBackend/.../config/DataInitializer.java

COMMANDLINERUNNER

Un bean au comportement particulier : au lieu d'attendre d'être appelé, il **s'exécute automatiquement** une fois le conteneur prêt. Sert à peupler la base au démarrage avec des données de démo (utile pour la présentation).

```
0.  @Component
1.  public class DataInitializer implements CommandLineRunner {
2.
3.      private static final Logger log = LoggerFactory.getLogger(DataInitializer.class);
4.
5.      private final AuteurRepository auteurRepo;
6.      private final LivreRepository livreRepo;
7.      private final UtilisateurRepository userRepo;
8.      private final EmpruntRepository empruntRepo;
9.      private final ReservationRepository reservationRepo;
0.      private final PasswordEncoder encoder;
1.
2.      public DataInitializer(AuteurRepository auteurRepo, LivreRepository livreRepo, UtilisateurRepository userRepo,
3.      EmpruntRepository empruntRepo, ReservationRepository reservationRepo, PasswordEncoder encoder)
4.      {
5.          this.auteurRepo = auteurRepo;
6.          this.livreRepo = livreRepo;
7.          this.userRepo = userRepo;
8.          this.empruntRepo = empruntRepo;
9.          this.reservationRepo = reservationRepo;
0.          this.encoder = encoder;
1.      }
2.  }
```

```
2.      @Override
3.      public void run(String... args) {
4.          if (userRepo.count() > 0) {
5.              log.info("Base déjà initialisée – aucune donnée de démo insérée.");
6.              return;
7.          }
8.          log.info("Initialisation des données de démonstration...");
9.
10.         [... construction des auteurs, livres, utilisateurs, emprunts et réservations de démo, lignes 50-97 ...]
11.
12.         log.info("Données de démo insérées. Comptes : admin@universite.sn / biblio@universite.sn / etudiant@universite.sn");
13.     }
```

L.20-21

Ici `@Component` suffit (contrairement à `JwtAuthFilter`) car le constructeur ne demande que des types déjà connus du conteneur (repositories Spring Data générés automatiquement, `PasswordEncoder` défini dans `BeansConfig`). `implements CommandLineRunner` est l'interface que Spring Boot détecte pour exécuter `run()` juste après le démarrage complet du contexte.

L.32-33

Constructeur à **6 dépendances injectées** — bel exemple pour illustrer que l'injection par constructeur scale sans effort : Spring résout chacune par type, peu importe le nombre.

L.44-47

Garde d'idempotence : si des utilisateurs existent déjà, on ne réinsère rien. C'est pour ça qu'en profil `dev` (base H2 recrée à chaque démarrage) les données de démo réapparaissent toujours, alors qu'en profil par défaut (MySQL persistante) elles ne sont insérées qu'une seule fois, au tout premier lancement.

QUESTION PROBABLE DU JURY

« Comment les comptes de démonstration sont-ils créés ? » — Par ce `CommandLineRunner`, exécuté une fois le conteneur Spring entièrement initialisé. Les mots de passe ne sont jamais stockés en clair : chaque appel `utilisateur(...)` passe par `encoder.encode(motDePasse)` (BCrypt, voir 1.2) avant `save()`.

Une autre façon d'injecter — pas des services, mais de la **configuration externalisée** (fichier `application.properties`) sous forme d'un bean typé.

```
1. package com.projet.bibliotheque.config;
2.
3. import org.springframework.boot.context.properties.ConfigurationProperties;
4. import org.springframework.stereotype.Component;
5.
6. import java.math.BigDecimal;
7.
8. /** Règles métier configurables (préfixe "biblio" dans application.properties). */
9. @Component
10. @ConfigurationProperties(prefix = "biblio")
1. public class BiblioProperties {
2.
3.     private final Emprunt emprunt = new Emprunt();
4.     private final Penalite penalite = new Penalite();
5.     private final Reservation reservation = new Reservation();
6.
7.     public Emprunt getEmprunt() { return emprunt; }
8.     public Penalite getPenalite() { return penalite; }
9.     public Reservation getReservation() { return reservation; }
10.
11.     public static class Emprunt {
12.         private int dureeJours = 14;
13.         private int prolongationJours = 7;
```

```

4.     private int maxSimultanes = 3;

5.     // + getters/setters standards pour les 3 champs

6. }

7. // classes internes Penalite{tarifJournalier} et Reservation{expirationHeures} - même principe

8. }

```

L.9-10

Combinaison classique : `@Component` pour que la classe devienne un bean injectable, `@ConfigurationProperties(prefix="biblio")` pour que Spring Boot fasse correspondre automatiquement chaque propriété `biblio.emprunt.duree-jours=14` du fichier de config à `emprunt.setDureeJours(14)` (conversion kebab-case → camelCase automatique).

L.20-25

Classes internes statiques imbriquées = une façon de grouper des propriétés liées sous un même préfixe (`biblio.emprunt.*`, `biblio.penalite.*`, `biblio.reservation.*`). Toute classe métier (ex: `EmpruntService`) peut injecter `BiblioProperties` dans son constructeur et lire `props.getEmprunt().getDureeJours()` plutôt que coder « 14 » en dur.

QUESTION PROBABLE DU JURY

« Pourquoi pas simplement plusieurs `@Value("${biblio.emprunt.duree-jours}")` ? » — Ça fonctionnerait aussi, mais il faudrait répéter l'annotation dans chaque classe qui a besoin d'une règle métier, sans validation de type groupée ni auto-complétion IDE. `@ConfigurationProperties` centralise, type et documente toutes les règles métier en un seul endroit.

1.6 — SYNTHÈSE IOC / DI À RETENIR

Le pattern est identique partout dans le projet, backend comme frontend : chaque `@RestController`, chaque `@Service`, chaque `@Component` déclare ses dépendances comme des champs `private final`, reçues via un unique constructeur — jamais de `new` pour un collaborateur, jamais de couplage direct. On le voit dans `AuthController` (5 dépendances), `LivreService` (5 dépendances), `UserService` (5 dépendances), etc. Coté Angular, l'équivalent est la fonction `inject()` (voir 4.1).

Portée par défaut d'un bean Spring : singleton. Une seule instance de `LivreService` existe dans toute l'application, partagée par toutes les requêtes HTTP simultanées — d'où l'importance de ne jamais stocker d'état mutable propre à une requête dans un champ de service.

2 JPA & Persistance

Comment les objets Java deviennent des lignes en base

JPA (via Hibernate) fait correspondre des classes Java annotées (`@Entity`) à des tables, et Spring Data JPA génère automatiquement l'implémentation des repositories à partir de simples interfaces. Les quatre entités ci-dessous couvrent volontairement les trois types de relations du projet : **@ManyToOne** (Emprunt→Livre), **@ManyToMany** bidirectionnelle (Livre↔Auteur) et, pour mémoire, **@OneToOne** (Penalite→Emprunt, voir 2.6).

BuBackend/.../model/Utilisateur.java

@ENTITY

L'entité racine de l'authentification : un compte = un rôle = un mot de passe haché.

```
1. package com.projet.bibliotheque.model;
2.
3. import jakarta.persistence.*;
4.
5. import java.time.LocalDate;
6.
7. @Entity
8. @Table(name = "utilisateurs")
9. public class Utilisateur {
10.
11.     public enum Role { ETUDIANT, BIBLIOTHECAIRE, ADMIN }
12.
13.     @Id
14.     @GeneratedValue(strategy = GenerationType.IDENTITY)
15.     private Long id;
16.
17.     @Column(nullable = false, length = 80)
18.     private String nom;
19.
20.     @Column(unique = true, nullable = false)
21.     private String email;
22.
23.     @Column(nullable = false)
24.     private String motDePasse; // toujours stocké hashé (BCrypt)
```

```
5.
6.     @Enumerated(EnumType.STRING)
7.
8.     @Column(nullable = false)
9.
10.    private Role role = Role.ETUDIANT;
11.
12.
13.
14.    @Column(nullable = false)
15.
16.    private boolean actif = true;
17.
18.
19.
20.
21.    @Column(nullable = false)
22.
23.    private LocalDate dateInscription = LocalDate.now();
24.
25.
26.
27.
28.    public Utilisateur() { }
29.
30.    // + getters/setters standards pour chaque champ (id, nom, email, motDePasse, role, actif, dateInscription)
31.
32. }
```

L.7-8

`@Entity` déclare la classe comme persistante ; `@Table(name=...)` fixe le nom réel de la table (sinon Hibernate utiliserait le nom de la classe).

L.11

Un `enum` Java imbriqué modélise les 3 rôles — utilisé aussi bien côté persistance que côté sécurité (`hasRole(...)` , voir 3.3).

L.13-14

`@Id` marque la clé primaire. `GenerationType.IDENTITY` délègue la génération à la base (colonne `AUTO_INCREMENT` côté MySQL) — Hibernate ne calcule rien lui-même, il lit l'id généré après l' `INSERT` .

L.20

`@Column(unique = true)` ajoute une contrainte d'unicité SQL sur l'email — la vérification applicative (`existsByEmail` , voir 2.5) évite de laisser remonter une exception SQL brute, mais la contrainte base reste le filet de sécurité final.

L.26

`@Enumerated(EnumType.STRING)` stocke le nom littéral de l'enum (`"ADMIN"`) plutôt que son indice (`EnumType.ORDINAL` stockerait `2`). Plus lisible en base et surtout insensible à un réordonnement futur de l'enum.

PIÈGE À CONNAÎTRE

Cette entité n'est **jamais** renvoyée telle quelle par l'API (elle contient `motDePasse` , même haché). Voir `DtoMapper.toUtilisateurDto()` (3.11) : la conversion en `UtilisateurDto` exclut délibérément ce champ.

Le cœur du catalogue, et l'entité qui porte la relation N,N avec les auteurs.

```
1. package com.projet.bibliotheque.model;
2.
3. import jakarta.persistence.*;
4.
5. import java.util.ArrayList;
6. import java.util.List;
7.
8. @Entity
9. @Table(name = "livres")
0. public class Livre {
1.
2.     @Id
3.     @GeneratedValue(strategy = GenerationType.IDENTITY)
4.     private Long id;
5.
6.     @Column(nullable = false)
7.     private String titre;
8.
9.     @Column(unique = true, length = 13)
0.     private String isbn;
1.
2.     private Integer anneePublication;
3.
4.     @Column(length = 60)
```

```
5.     private String categorie;
6.
7.     @Column(nullable = false)
8.     private Integer stockTotal = 1;
9.
10.    @Column(nullable = false)
11.    private Integer stockDisponible = 1;
12.
13.    // Couverture en base64 (data URI). Longueur portable : Hibernate mappe en
14.    // (MEDIUM/LONG)TEXT sur MySQL et en grand VARCHAR/CLOB sur H2 – pas de DDL brut.
15.
16.    @Column(name = "image_url", length = 5_000_000)
17.    private String imageUrl;
18.
19.    @Column(length = 2000)
20.    private String description;
21.
22.    // Relation N,N : un livre peut avoir plusieurs auteurs et inversement
23.
24.    @ManyToMany(fetch = FetchType.LAZY)
25.    @JoinTable(name = "livre_auteur",
26.
27.        joinColumns = @JoinColumn(name = "livre_id"),
28.
29.        inverseJoinColumns = @JoinColumn(name = "auteur_id"))
30.    private List<Auteur> auteurs = new ArrayList<>();
31.
32.
33.    public Livre() { }
34.
35.    // + getters/setters standards pour chaque champ, dont getAuteurs()/setAuteurs()
36.
37. }
```

L.19

`@Column(unique = true, length = 13)` — un ISBN-13 ; l'unicité empêche deux fiches livre avec le même ISBN (vérifié aussi en amont dans `LivreService.creer()` , voir 3.11, pour renvoyer un 409 propre plutôt qu'une erreur SQL).

L.35-38

La couverture est stockée en base64 directement en base (pas de stockage fichier séparé) — choix simple pour un TP. Le commentaire explique pourquoi on ne force pas un type SQL précis : Hibernate choisit le type colonne adapté au dialecte actif (H2 vs MySQL) à partir de la seule `length` .

L.42-46

Le bloc clé de la relation N,N. `@ManyToMany` seule ne suffirait pas à nommer la table de jointure : `@JoinTable` précise explicitement `livre_auteur` avec ses deux clés étrangères. `Livre` est le **côté propriétaire** de la relation (celui qui porte `@JoinTable`) : c'est lui qui pilote les `INSERT / DELETE` dans la table de jointure quand on appelle `setAuteurs(...)` .

L.42

`FetchType.LAZY` : la liste des auteurs n'est chargée depuis la base **que si on y accède explicitement** (`livre.getAuteurs()`), pas automatiquement à chaque `findById` . Évite de charger des données inutiles si on ne veut qu'un titre.

QUESTION PROBABLE DU JURY

« Que se passe-t-il si vous chargez un livre en dehors d'une transaction et que vous accédez à `getAuteurs()` ? » — Normalement une `LazyInitializationException` (la session Hibernate est fermée). Le projet l'évite en convertissant systématiquement l'entité en DTO **à l'intérieur** de la méthode `@Transactional` du service (voir 2.7, `open-in-view=false`), jamais dans le contrôleur.

Le miroir de `Livre` : même relation, vue depuis l'autre bout.

```
1. package com.projet.bibliotheque.model;
2.
3. import jakarta.persistence.*;
4.
5. import java.util.ArrayList;
6. import java.util.List;
7.
8. @Entity
9. @Table(name = "auteurs")
0. public class Auteur {
1.
2.     @Id
3.     @GeneratedValue(strategy = GenerationType.IDENTITY)
4.     private Long id;
5.
6.     @Column(nullable = false, length = 80)
7.     private String nom;
8.
9.     @Column(nullable = false, length = 80)
0.     private String prenom;
1.
2.     @Column(length = 60)
3.     private String nationalite;
4.
```

```
5. @ManyToMany(mappedBy = "auteurs", fetch = FetchType.LAZY)
6.
7.
8. private List<Livres> livres = new ArrayList<>();
9.
10.
11.
12. public Auteur() { }
13.
14.
15.
16. public Auteur(String nom, String prenom, String nationalite) {
17.
18.     this.nom = nom;
19.
20.     this.prenom = prenom;
21.
22.     this.nationalite = nationalite;
23.
24. }
25.
26. // + getters/setters standards, dont getLivres()/setLivres()
27.
28. }
```

L.25-26

`mappedBy = "auteurs"` désigne le champ `auteurs` de la classe `Livres` comme propriétaire réel de la relation. Conséquence directe : **aucune** `@JoinTable` ici, et modifier `auteur.getLivres().add(...)` tout seul **n'écrit rien en base** — il faut passer par `livre.setAuteurs(...)` côté propriétaire (c'est exactement ce que fait `LivresService.appliquer()`, voir 3.11).

L.31-35

Constructeur additionnel à 3 arguments : pratique utilisé uniquement par `DataInitializer` pour créer les auteurs de démo en une ligne (`new Auteur("Hugo", "Victor", "Française")`).

La transaction métier centrale : qui a emprunté quel livre, et jusqu'à quand.

```
1. package com.projet.bibliotheque.model;
2.
3. import jakarta.persistence.*;
4.
5. import java.time.LocalDate;
6.
7. @Entity
8. @Table(name = "emprunts")
9. public class Emprunt {
10.
11.     public enum Statut { EN_COURS, RENDU, EN_RETARD }
12.
13.     @Id
14.     @GeneratedValue(strategy = GenerationType.IDENTITY)
15.     private Long id;
16.
17.     @ManyToOne(fetch = FetchType.LAZY, optional = false)
18.     @JoinColumn(name = "utilisateur_id", nullable = false)
19.     private Utilisateur utilisateur;
20.
21.     @ManyToOne(fetch = FetchType.LAZY, optional = false)
22.     @JoinColumn(name = "livre_id", nullable = false)
23.     private Livre livre;
24.
```

```
5.     @Column(nullable = false)
6.
7.     private LocalDate dateEmprunt = LocalDate.now();
8.
9.     @Column(nullable = false)
10.
11.     private LocalDate dateRetourPrevue;
12.
13.     @Column(nullable = false)
14.
15.     private LocalDate dateRetourEffective;
16.
17.     @Enumerated(EnumType.STRING)
18.
19.     @Column(nullable = false)
20.
21.     private Statut statut = Statut.EN_COURS;
22.
23.
24.     @Column(nullable = false)
25.
26.     private boolean prolonge = false;
27.
28.
29.     public Emprunt() { }
30.
31.     // + getters/setters standards pour chaque champ
32. }
```

L.16-18 et L.20-22

Deux `@ManyToOne` : plusieurs emprunts peuvent référencer le même utilisateur, et plusieurs emprunts peuvent référencer le même livre. `optional = false` + `nullable = false` imposent qu'un emprunt ait toujours les deux — cohérent avec le domaine (pas d'emprunt sans emprunteur ni sans livre).

L.18, L.22

`@JoinColumn(name=...)` nomme la colonne de clé étrangère physique (`utilisateur_id`, `livre_id`) dans la table `emprunts` — à la différence du N,N de `Livre`, un `@ManyToOne` ne crée pas de table intermédiaire, juste une colonne FK.

L.31

`dateRetourEffective` est le champ pivot de toute la logique métier : `null` = pas encore rendu (utilisé par `EmpruntRepository.countByDateRetourEffectiveIsNull()` , 2.5), une date = rendu.

QUESTION PROBABLE DU JURY

« Pourquoi stocker à la fois un `statut` (enum) **et** déduire le retard depuis les dates ? » — Le `statut` est mis à jour de façon asynchrone par `SchedulerService` (tâche planifiée, `@Scheduled`) ; en attendant son prochain passage, `dateRetourEffective == null && dateRetourPrevue.isBefore(now())` donne un retard « en temps réel » sans attendre le prochain lot — vous le voyez utilisé dans `DtoMapper.toEmpruntDto()` (3.11) et `EmpruntRepository.countByDateRetourEffectiveIsNullAndDateRetourPrevueBefore(...)`.

Aucune de ces interfaces n'a d'implémentation écrite à la main : Spring Data génère un **proxy dynamique** au démarrage à partir du nom de la méthode et/ou de l'annotation `@Query`.

```
1. package com.projet.bibliotheque.repository;
2.
3. import com.projet.bibliotheque.model.Utilisateur;
4. import org.springframework.data.jpa.repository.JpaRepository;
5.
6. import java.util.Optional;
7.
8. public interface UtilisateurRepository extends JpaRepository<Utilisateur, Long> {
9.
10.     Optional<Utilisateur> findByEmail(String email);
11.
12.     boolean existsByEmail(String email);
13.
14.     long countByRole(Utilisateur.Role role);
15. }
```

L.8

`JpaRepository<Utilisateur, Long>` fournit déjà gratuitement `save()`, `findById()`, `findAll()`, `delete()`, `count()` ... (type d'entité, type de la clé primaire).

L.10

`findByEmail(String email)` : une **méthode dérivée**. Spring Data parse le nom (`findBy` + `Email`) et le traduit en JPQL équivalent à `SELECT u FROM Utilisateur u WHERE u.email = :email` — sans une seule ligne de SQL écrite. Le retour en `Optional` force l'appelant à gérer explicitement le cas « non trouvé » (voir `UserService.parEmail()`, 3.6, avec `.orElseThrow(...)`).

Requêtes plus riches — LivreRepository : dès qu'un nom de méthode dérivée deviendrait imbuvable, on bascule sur du JPQL explicite avec **@Query**.

```
0. public interface LivreRepository extends JpaRepository<Livre, Long> {
1.
2.     boolean existsByIsbn(String isbn);
3.
4.     List<Livre> findByStockDisponibleGreaterThan(int stock);
5.
6.     // Recherche multi-critères : titre, catégorie ou nom/prénom d'un des auteurs
7.     @Query("""
8.         SELECT DISTINCT l FROM Livre l LEFT JOIN l.auteurs a
9.         WHERE LOWER(l.titre) LIKE LOWER(CONCAT('%', :q, '%'))
0.         OR LOWER(l.categorie) LIKE LOWER(CONCAT('%', :q, '%'))
1.         OR LOWER(a.nom) LIKE LOWER(CONCAT('%', :q, '%'))
2.         OR LOWER(a.prenom) LIKE LOWER(CONCAT('%', :q, '%'))
3.         ORDER BY l.titre
4.     """)
5.     List<Livre> rechercher(@Param("q") String query);
6.
7.     @Query("SELECT l FROM Livre l JOIN l.auteurs a WHERE a.id = :auteurId ORDER BY l.titre")
8.     List<Livre> findByAuteurId(@Param("auteurId") Long auteurId);
9. }
```

L.14

findByStockDisponibleGreaterThan(int) : autre méthode dérivée, traduite en **WHERE stockDisponible > ?**.

L.17-24

JPQL (langage orienté objet : on écrit `Livre l, l.auteurs` — des entités et leurs champs, pas des tables/colonnes SQL). Le `""` est un *text block* Java 17 (chaîne multi-lignes). `LEFT JOIN l.auteurs a` permet de chercher aussi par nom d'auteur ; `DISTINCT` évite les doublons de livre quand plusieurs de ses auteurs correspondent au filtre (fan-out de jointure) ; `:q` est un paramètre nommé, relié à `@Param("q")` sur le paramètre Java.

L.25

`@Param("q") String query` : le nom Java du paramètre (`query`) n'a pas besoin de correspondre au nom JPQL (`:q`) grâce à `@Param` .

Agrégations pour le dashboard — `EmpruntRepository` :

```
5. @Query("""
6.     SELECT l.id, l.titre, COUNT(e) AS nb FROM Emprunt e JOIN e.livre l
7.     GROUP BY l.id, l.titre ORDER BY nb DESC
8.     """)
9. List<Object[]> topLivresEmpruntes(org.springframework.data.domain.Pageable pageable);
```

L.45-49

Cette requête ne sélectionne pas une entité complète mais 3 colonnes hétérogènes (`id` , `titre` , un `COUNT` calculé) : Hibernate ne peut plus la mapper vers `Livre` , il renvoie donc une liste de `Object[]` — un tableau par ligne de résultat, à caster manuellement côté service (`StatsService`).

L.49

`Pageable` en paramètre : Spring Data l'utilise automatiquement pour ajouter `LIMIT` /tri à la requête (ex: `PageRequest.of(0, 5)` pour le top 5), sans changer le JPQL.

REPOSITORY

SIGNATURES NOTABLES

AuteurRepository

`findByNomContainingIgnoreCaseOrPrenomContainingIgnoreCase(...)`,
`findByNomAndPrenom(...)` — deux méthodes dérivées de plus, combinant
OR / AND et une recherche insensible à la casse (`ContainingIgnoreCase`).

ReservationRepository

`findByLivreIdAndStatutOrderByDateReservationAsc(...)` matérialise la file
d'attente FIFO ; `countByLivreIdAndStatutAndDateReservationBefore(...)`
calcule la position d'une réservation dans la file en comptant celles qui sont plus
anciennes.

EmpruntRepository

`existsByUtilisateurId(...)` / `existsByLivreId(...)` — utilisées avant une
suppression pour préserver l'intégrité référentielle applicative (voir 3.11).

Même squelette que `Emprunt` (`@Id / @GeneratedValue(IDENTITY)` , `@Enumerated(STRING)` ,
accesseurs standards) — non détaillées ligne par ligne ici, mais utiles pour compléter le tableau des
types de relations JPA du projet si le jury demande un exemple de `@OneToOne` .

ENTITÉ	RELATIONS	PARTICULARITÉ
Reservation	2× <code>@ManyToOne</code> (Utilisateur, Livre)	Enum <code>Statut</code> à 5 valeurs (<code>EN_ATTENTE</code> , <code>DISPONIBLE</code> , <code>CONFIRMEE</code> , <code>ANNULEE</code> , <code>EXPIREE</code> qui pilote la file d'attente FIFO — triée par <code>dateReservation</code> .
Penalite	<code>@OneToOne(optional = false)</code> vers <code>Emprunt</code> , avec <code>@JoinColumn(unique = true)</code>	Le seul <code>@OneToOne</code> du modèle : une pénalité correspond à exactement un emprunt en retard. <code>montant</code> est un <code>BigDecimal(precision=10, scale=2)</code> — jamais un <code>double</code> pour de l'argent.
Notification	3× <code>@ManyToOne</code> (Utilisateur obligatoire ; Reservation et Emprunt optionnels , sans <code>nullable=false</code>)	Les FK vers Reservation/Emprunt sont nullables : une notification peut être un simple message (<code>INFO</code>) sans lien vers une ressource précise.



QUESTION PROBABLE DU JURY

« Donnez un exemple de `@OneToOne` dans votre modèle. » — `Penalite.emprunt` : `@OneToOne(fetch = FetchType.LAZY, optional = false)` avec `@JoinColumn(name = "emprunt_id", nullable = false, unique = true)` . C'est le `unique = true` sur la colonne de jointure qui transforme une relation qui ressemblerait sinon à un `@ManyToOne` en véritable un-à-un au niveau base de données.

application.properties – clés JPA

CONFIG

CLÉ	VALEUR	EFFET
spring.jpa.hibernate.ddl-auto	update (défaut/prod) · create-drop (dev)	Prod : Hibernate ajoute les colonnes/tables manquantes sans jamais supprimer de données existantes. Dev : le schéma H2 est recréé puis détruit à chaque redémarrage — d'où le réamorçage systématique par <code>DataInitializer</code> en dev.
spring.jpa.open-in-view	false	Sans OSIV (Open Session In View), la session Hibernate se ferme à la fin de la méthode <code>@Transactional</code> du service — pas à la fin de la requête HTTP. D'où l'obligation de convertir en DTO à l'intérieur du service (voir 2.2), jamais dans le contrôleur.
spring.jpa.show-sql	false (prod) · true (dev)	Utile en dev pour voir le SQL réellement généré par Hibernate dans la console — bon réflexe à montrer en live si le jury demande une preuve.

3 API REST & JWT

Authentification stateless, de bout en bout

C'est la partie la plus susceptible d'être creusée en profondeur : le professeur a explicitement mentionné « token JWT ». L'ordre ci-dessous suit le trajet réel d'un jeton : sa **fabrication** (JwtService), son **interception** à chaque requête (JwtAuthFilter), les **règles** qui décident qui a le droit de faire quoi (SecurityConfig), puis les contrôleurs qui en dépendent.

BuBackend/.../security/JwtService.java

FABRIQUE & VÉRIFIE LES TOKENS

Toute la cryptographie du projet tient dans ce fichier de 59 lignes, construit autour de la bibliothèque `io.jsonwebtoken` (JJWT).

```
1. package com.projet.bibliotheque.security;
2.
3. import io.jsonwebtoken.Claims;
4. import io.jsonwebtoken.JwtException;
5. import io.jsonwebtoken.Jwts;
6. import io.jsonwebtoken.security.Keys;
7. import org.springframework.beans.factory.annotation.Value;
8. import org.springframework.security.core.userdetails.UserDetails;
9. import org.springframework.stereotype.Component;
10.
11. import javax.crypto.SecretKey;
12. import java.nio.charset.StandardCharsets;
13. import java.util.Date;
14.
15. @Component
16. public class JwtService {
17.
18.     @Value("${jwt.secret}")
19.     private String secret;
20.
21.     @Value("${jwt.expiration}")
22.     private long expiration;
23.
```

```
4.     private SecretKey cle() {
5.         return Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
6.     }
7.
8.     public String genererToken(UserDetails user, String role) {
9.         return Jwts.builder()
10.            .subject(user.getUsername())
11.            .claim("role", role)
12.            .issuedAt(new Date())
13.            .expiration(new Date(System.currentTimeMillis() + expiration))
14.            .signWith(cle())
15.            .compact();
16.     }
17.
18.     public String extraireUsername(String token) {
19.         return parse(token).getSubject();
20.     }
21.
22.     public boolean valider(String token, UserDetails user) {
23.         try {
24.             Claims claims = parse(token);
25.             return claims.getSubject().equals(user.getUsername())
26.                && claims.getExpiration().after(new Date());
27.         } catch (JwtException | IllegalArgumentException e) {
28.             return false;
29.         }
30.     }
31. }
```

```
2.     private Claims parse(String token) {  
3.         return Jwts.parser()  
4.             .verifyWith(cle())  
5.             .build()  
6.             .parseSignedClaims(token)  
7.             .getPayload();  
8.     }  
9. }
```

L.18-19, L.21-22

`@Value("${jwt.secret}")` injecte directement une **valeur scalaire** lue dans `application.properties` (pas un bean complexe : ici l'injection de champ est acceptable, contrairement aux dépendances objet qui passent par le constructeur ailleurs dans le projet). `jwt.secret` est la clé symétrique, `jwt.expiration` la durée de vie en millisecondes (86 400 000 = 24h, voir 5).

L.24-26

`cle()` convertit la chaîne secrète en `SecretKey` HMAC-SHA. C'est une clé **symétrique** : la même clé sert à signer et à vérifier — elle ne doit jamais fuiter côté client.

L.28-36

`genererToken` construit le JWT : `subject` = l'email (identifiant unique), `claim("role", role)` = une donnée métier ajoutée dans le payload, `issuedAt / expiration` = horodatages, `signWith(cle())` = signature HMAC, `compact()` = sérialisation finale en 3 segments `header.payload.signature` séparés par des points.

L.42-49

`valider` revérifie **deux choses** : que le token appartient bien à l'utilisateur chargé (`getSubject().equals(...)`) et qu'il n'est pas expiré. Toute exception de parsing (signature invalide, JSON corrompu, format incorrect) est interceptée et transformée en simple `false` — jamais une exception ne remonte telle quelle.

L.52-58

`parse()` est le seul endroit où la signature est réellement vérifiée : `verifyWith(cle())` configure la vérification, `parseSignedClaims(token)` lève une `JwtException` si la signature ne correspond pas au contenu — **c'est ce qui rend un payload modifié détectable** (voir Test 2, section 6).

QUESTION PROBABLE DU JURY

« Le contenu du token est-il chiffré ? » — **Non, seulement signé.** Le payload (`subject` , `role` , `dates`) est encodé en Base64URL — lisible par n'importe qui sur jwt.io, y compris le rôle de l'utilisateur. La signature garantit uniquement l'**intégrité** (impossible de modifier le contenu sans invalider la signature), pas la confidentialité. C'est pourquoi le payload ne contient jamais le mot de passe.

Le filtre qui s'exécute sur **chaque** requête HTTP entrante pour transformer un header **Authorization** en identité Spring Security.

```
1. package com.projet.bibliotheque.security;
2.
3. import jakarta.servlet.FilterChain;
4. import jakarta.servlet.ServletException;
5. import jakarta.servlet.http.HttpServletRequest;
6. import jakarta.servlet.http.HttpServletResponse;
7. import org.springframework.lang.NonNull;
8. import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
9. import org.springframework.security.core.context.SecurityContextHolder;
0. import org.springframework.security.core.userdetails.UserDetails;
1. import org.springframework.security.core.userdetails.UserDetailsService;
2. import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
3. import org.springframework.web.filter.OncePerRequestFilter;
4.
5. import java.io.IOException;
6.
7. public class JwtAuthFilter extends OncePerRequestFilter {
8.
9.     private final JwtService jwtService;
0.     private final UserDetailsService userDetailsService;
1.
2.     public JwtAuthFilter(JwtService jwtService, UserDetailsService userDetailsService) {
3.         this.jwtService = jwtService;
```

```
4.         this.userDetailsService = userDetailsService;
5.     }
6.
7.     @Override
8.     protected void doFilterInternal(@NonNull HttpServletRequest request,
9.                                     @NonNull HttpServletResponse response,
10.                                    @NonNull FilterChain filterChain) throws ServletException, IOException {
11.         String header = request.getHeader("Authorization");
12.
13.         if (header == null || !header.startsWith("Bearer ")) {
14.             filterChain.doFilter(request, response);
15.             return;
16.         }
17.
18.         String token = header.substring(7);
19.
20.         try {
21.             String email = jwtService.extraireUsername(token);
22.
23.             if (email != null && SecurityContextHolder.getContext().getAuthentication() == null) {
24.                 UserDetails user = userDetailsService.loadUserByUsername(email);
25.
26.                 if (jwtService.valider(token, user)) {
27.                     var auth = new UsernamePasswordAuthenticationToken(user, null, user.getAuthorities());
28.
29.                     auth.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
30.
31.                     SecurityContextHolder.getContext().setAuthentication(auth);
32.                 }
33.             }
34.
35.         } catch (Exception e) {
36.
37.             // Token invalide/expiré : on laisse la requête continuer sans authentification
38.
39.             SecurityContextHolder.clearContext();
40.
41.         }
```



```
2.         filterChain.doFilter(request, response);
3.     }
4. }
```

L.17

`extends OncePerRequestFilter` — classe utilitaire Spring qui garantit que `doFilterInternal` ne s'exécute **qu'une fois** par requête (même si la requête traverse plusieurs fois la chaîne de filtres internes).

L.29-32

Pas de header `Authorization`, ou pas au format `Bearer <token>` → on laisse passer la requête **sans authentifier** (utile pour les endpoints publics comme `GET /api/livres`). C'est `authorizeHttpRequests` plus loin dans la chaîne qui décidera si l'absence d'authentification est un problème.

L.37

`header.substring(7)` retire le préfixe littéral `"Bearer "` (7 caractères) pour isoler le token brut.

L.40

Double garde : `email != null` (le token a pu être parsé) **et** `getAuthentication() == null` (on n'écrase pas une authentification déjà posée plus tôt dans la chaîne).

L.41-45

Le cœur du mécanisme : on recharge l'utilisateur depuis la base (`loadUserByUsername`, voir 3.6), on revalide le token contre lui (signature + expiration + correspondance de subject), puis on construit un `UsernamePasswordAuthenticationToken` — malgré son nom, il ne transporte **aucun mot de passe** ici (2^e argument `null`), seulement l'utilisateur et ses `authorities` (rôles).

L.46

`SecurityContextHolder.getContext().setAuthentication(auth)` : c'est **cette ligne précise** qui rend la requête « authentifiée » pour tout le reste du traitement Spring (règles d'autorisation, `@PreAuthorize`, `SecurityContextHolder` lu plus tard par `CurrentUser`).

L.48-51

N'importe quelle exception (signature invalide, token expiré, utilisateur supprimé...) est interceptée : le contexte est explicitement vidé et la requête continue **sans** authentification — jamais d'erreur 500 pour un token cassé, c'est la règle d'autorisation en aval qui renverra proprement un 401.

QUESTION PROBABLE DU JURY

« Ce filtre est-il un bean Spring comme les autres ? » — Oui et non : la classe elle-même n'est **pas** annotée `@Component` (elle a besoin d'arguments de constructeur non triviaux), mais elle est enregistrée comme bean via la méthode `@Bean jwtAuthFilter()` de `SecurityConfig` (voir 1.3) — un rappel qu'loC/DI et JWT se rejoignent concrètement à cet endroit précis du code.

BuBackend/.../config/SecurityConfig.java

SECURITYFILTERCHAIN

Le règlement complet : qui peut appeler quoi, avec ou sans authentification, et que renvoyer quand ce n'est pas le cas.

```
6. @Bean
7. public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
8.     http
9.         .cors(cors -> cors.configurationSource(corsConfigurationSource()))
10.        .csrf(csrf -> csrf.disable())
11.        .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
12.        .authorizeHttpRequests(auth -> auth
13.            // Consultation publique du catalogue
14.            .requestMatchers(HttpMethod.GET, "/api/livres/**").permitAll()
15.            .requestMatchers(HttpMethod.GET, "/api/auteurs/**").permitAll()
16.            // Authentification
17.            .requestMatchers("/api/auth/register", "/api/auth/login").permitAll()
18.            // Console H2 (profil dev)
19.            .requestMatchers("/h2-console/**").permitAll()
20.            // Gestion du catalogue : bibliothécaire ou admin
21.            .requestMatchers(HttpMethod.POST, "/api/livres/**", "/api/auteurs/**")
22.            .hasAnyRole("BIBLIOTHECAIRE", "ADMIN")
23.            .requestMatchers(HttpMethod.PUT, "/api/livres/**", "/api/auteurs/**")
24.            .hasAnyRole("BIBLIOTHECAIRE", "ADMIN")
25.            .requestMatchers(HttpMethod.DELETE, "/api/livres/**", "/api/auteurs/**")
26.            .hasAnyRole("BIBLIOTHECAIRE", "ADMIN")
27.            // Espace de gestion (dashboard, retours, pénalités, tous les emprunts...)
28.            .requestMatchers("/api/stats/**").hasAnyRole("BIBLIOTHECAIRE", "ADMIN")
```

```

9.         .requestMatchers("/api/admin/**").hasRole("ADMIN")
10.
11.         // Tout le reste nécessite d'être authentifié
12.
13.         .anyRequest().authenticated()
14.
15.     )
16.
17.     .headers(headers -> headers.frameOptions(frame -> frame.sameOrigin())) // console H2
18.
19.     .exceptionHandling(ex -> ex
20.
21.         // Non authentifié → 401 (et non 403) pour permettre au front de rediriger vers Le Login
22.
23.         .authenticationEntryPoint((request, response, e) ->
24.
25.             écrireErreur(response, HttpServletResponse.SC_UNAUTHORIZED,
26.
27.                 "Authentification requise"))
28.
29.         // Authentifié mais privilèges insuffisants → 403
30.
31.         .accessDeniedHandler((request, response, e) ->
32.
33.             écrireErreur(response, HttpServletResponse.SC_FORBIDDEN,
34.
35.                 "Accès refusé : privilèges insuffisants"))
36.
37.         )
38.
39.     .addFilterBefore(jwtAuthFilter(), UsernamePasswordAuthenticationFilter.class);
40.
41.     return http.build();
42. }

```

L.49

`csrf.disable()` : la protection CSRF protège les sessions basées sur **cookies** (envoyés automatiquement par le navigateur). Ici, l'authentification passe par un header `Authorization` ajouté explicitement en JavaScript (voir 4.2) — un site tiers ne peut pas forcer son envoi, donc CSRF n'est pas un vecteur d'attaque pertinent pour cette API stateless.

L.50

`SessionCreationPolicy.STATELESS` : Spring ne crée ni ne lit jamais de `HttpSession`. Chaque requête doit prouver seule son identité via le token — aucun état de connexion n'est gardé côté serveur, ce qui permettrait de scaler horizontalement sans partage de session.

L.52-70

`authorizeHttpRequests` : les règles sont évaluées **dans l'ordre**, la première qui matche gagne. D'où l'importance de mettre les règles les plus spécifiques (GET public sur `/api/livres/**`) avant les plus larges, et de terminer par `anyRequest().authenticated()` comme filet de sécurité par défaut.

L.68

`hasRole("ADMIN")` vs `hasAnyRole(...)` : Spring Security préfixe automatiquement par `ROLE_` en interne — c'est pour ça que l'enum métier stocke juste `"ADMIN"` (voir 2.1) et que `UserService` appelle `.roles(user.getRole().name())` sans préfixe (voir 3.6).

L.73-82

La distinction **401 vs 403** programmée explicitement : `authenticationEntryPoint` se déclenche quand la requête n'est pas authentifiée du tout (pas de token / token invalide) → 401 ; `accessDeniedHandler` se déclenche quand elle est authentifiée mais que le rôle ne correspond pas → 403. Le commentaire du code le dit noir sur blanc : cette distinction existe **pour que le frontend sache** quand rediriger vers `/login` (401, voir 4.2) et quand juste afficher une erreur (403).

L.83

`addFilterBefore(jwtAuthFilter(), UsernamePasswordAuthenticationFilter.class)` : insère `JwtAuthFilter` **avant** le filtre de login par formulaire classique de Spring Security (non utilisé ici, mais toujours présent par défaut dans la chaîne). Il faut que l'authentification par token soit posée tôt, avant que les règles `authorizeHttpRequests` ne soient évaluées.

Récapitulatif des règles d'accès :

ROUTE	MÉTHODE	ACCÈS
<code>/api/livres/**</code> , <code>/api/auteurs/**</code>	GET	Public (<code>permitAll</code>)
<code>/api/auth/register</code> , <code>/api/auth/login</code>	POST	Public (<code>permitAll</code>)
<code>/api/livres/**</code> , <code>/api/auteurs/**</code>	POST / PUT / DELETE	BIBLIOTHECAIRE ou ADMIN
<code>/api/stats/**</code>	toutes	BIBLIOTHECAIRE ou ADMIN
<code>/api/admin/**</code>	toutes	ADMIN uniquement
tout le reste (ex: <code>/api/emprunts/**</code>)	toutes	Authentifié (n'importe quel rôle valide)

CORS — l'origine Angular en développement :

```

4.     @Bean
5.     public CorsConfigurationSource corsConfigurationSource() {

```

```
6. CorsConfiguration config = new CorsConfiguration();
7. config.setAllowedOrigins(List.of("http://localhost:4200"));
8. config.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
9. config.setAllowedHeaders(List.of("*"));
0. config.setAllowCredentials(true);
1.
2. UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
3. source.registerCorsConfiguration("/**", config);
4. return source;
5. }
6. }
```

L.97

Seule l'origine `http://localhost:4200` (port par défaut du serveur de dev Angular) est autorisée à appeler l'API depuis un navigateur — une origine différente (autre port, autre domaine) serait bloquée par le navigateur, même avec un token valide.

BuBackend/.../security/CurrentUser.java

HELPER

Un petit utilitaire injecté dans les contrôleurs pour retrouver « qui suis-je ? » sans jamais faire confiance à un paramètre envoyé par le client.

```
1. package com.projet.bibliotheque.security;
2.
3. import com.projet.bibliotheque.model.Utilisateur;
4. import com.projet.bibliotheque.service.UserService;
5. import org.springframework.security.core.context.SecurityContextHolder;
6. import org.springframework.security.core.userdetails.UserDetails;
7. import org.springframework.stereotype.Component;
8.
9. /** Résout l'utilisateur authentifié à partir du contexte de sécurité (email → entité). */
0. @Component
1. public class CurrentUser {
2.
3.     private final UserService userService;
4.
5.     public CurrentUser(UserService userService) {
6.         this.userService = userService;
7.     }
8.
9.     public Utilisateur get() {
0.         var auth = SecurityContextHolder.getContext().getAuthentication();
1.         if (auth == null || !(auth.getPrincipal() instanceof UserDetails details)) {
2.             throw new org.springframework.security.access.AccessDeniedException("Non authentifié");
3.         }
```

```
4.         return userService.parEmail(details.getUsername());
5.     }
6.
7.     public Long id() {
8.         return get().getId();
9.     }
0. }
```

L.19-24

Relit exactement ce que `JwtAuthFilter` a posé en L.46 (3.2) dans le `SecurityContextHolder` — le principal est le `UserDetails` chargé depuis la base. On retourne l'entité `Utilisateur` complète en refaisant une lecture par email, plutôt que de faire confiance à un `id` fourni dans l'URL ou le corps de la requête.

QUESTION PROBABLE DU JURY

« Pourquoi `EmpruntController.emprunter()` (3.10) ne prend-il pas l'id de l'utilisateur en paramètre de la requête ? » — Exactement pour éviter qu'un étudiant emprunte un livre « *au nom* » d'un autre en changeant un id dans le JSON envoyé. `currentUser.id()` déduit l'utilisateur **uniquement** du token vérifié cryptographiquement, jamais d'une donnée fournie par le client.

BuBackend/.../controller/AuthController.java

@RestController

Les trois seuls endpoints publics ou quasi-publics de toute l'API : s'inscrire, se connecter, se relire.

```
1. package com.projet.bibliotheque.controller;
2.
3. import com.projet.bibliotheque.dto.*;
4. import com.projet.bibliotheque.model.Utilisateur;
5. import com.projet.bibliotheque.security.CurrentUser;
6. import com.projet.bibliotheque.security.JwtService;
7. import com.projet.bibliotheque.service.UserService;
8. import jakarta.validation.Valid;
9. import org.springframework.http.HttpStatus;
0. import org.springframework.http.ResponseEntity;
1. import org.springframework.security.authentication.AuthenticationManager;
2. import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
3. import org.springframework.security.core.userdetails.UserDetails;
4. import org.springframework.security.core.userdetails.UserDetailsService;
5. import org.springframework.web.bind.annotation.*;
6.
7. @RestController
8. @RequestMapping("/api/auth")
9. public class AuthController {
0.
1.     private final AuthenticationManager authManager;
2.     private final JwtService jwtService;
3.     private final UserService userService;
4.     private final UserDetailsService userDetailsService;
```

```
5.     private final CurrentUser currentUser;
6.
7.     public AuthController(AuthenticationManager authManager, JwtService jwtService, UserService userService,
8.                           UserDetailsService userDetailsService, CurrentUser currentUser) {
9.
10.        this.authManager = authManager;
11.
12.        this.jwtService = jwtService;
13.
14.        this.userService = userService;
15.
16.        this.userDetailsService = userDetailsService;
17.
18.        this.currentUser = currentUser;
19.    }
20.
21.
22.    @PostMapping("/register")
23.
24.    public ResponseEntity<AuthResponse> register(@Valid @RequestBody RegisterRequest req) {
25.
26.        userService.inscrire(req.nom(), req.email(), req.password());
27.
28.        // Connexion automatique après inscription
29.
30.        return ResponseEntity.status(HttpStatus.CREATED).body(authentifier(req.email(), req.password()));
31.    }
32.
33.
34.    @PostMapping("/login")
35.
36.    public ResponseEntity<AuthResponse> login(@Valid @RequestBody LoginRequest req) {
37.
38.        return ResponseEntity.ok(authentifier(req.email(), req.password()));
39.    }
40.
41.
42.    @GetMapping("/me")
43.
44.    public ResponseEntity<UtilisateurDto> me() {
45.
46.        Utilisateur u = currentUser.get();
47.
48.        return ResponseEntity.ok(new UtilisateurDto(u.getId(), u.getNom(), u.getEmail(),
```

```

2.         u.getRole().name(), u.isActif(), u.getDateInscription());
3.     }
4.
5.     private AuthResponse authentifier(String email, String password) {
6.         authManager.authenticate(new UsernamePasswordAuthenticationToken(email, password));
7.         UserDetails details = userDetailsService.loadUserByUsername(email);
8.         Utilisateur u = userService.parEmail(email);
9.         String token = jwtService.genererToken(details, u.getRole().name());
0.         return new AuthResponse(token, "Bearer", u.getId(), u.getEmail(), u.getNom(), u.getRole().name());
1.     }
2. }

```

L.17-18

`@RestController` = `@Controller` + `@ResponseBody` : chaque valeur retournée par une méthode est directement sérialisée en JSON dans le corps de la réponse (pas de résolution de vue HTML).

`@RequestMapping("/api/auth")` préfixe toutes les routes du contrôleur.

L.27-33

Constructeur à 5 dépendances — encore une fois, aucune n'est instanciée manuellement.

L.36-41

`register` : `@Valid` déclenche la validation Bean Validation sur `RegisterRequest` (voir 3.7) **avant** même d'entrer dans le corps de la méthode — en cas d'échec, `GlobalExceptionHandler` intercepte (voir 3.8).

Après création du compte, on appelle directement `authentifier(...)` : l'utilisateur est **connecté automatiquement** après inscription, pas besoin d'un second appel `/login`. Code retour `201 CREATED`.

L.48-53

`/me` lit l'utilisateur via `currentUser.get()` (3.4) — jamais un id fourni par le client — puis reconstruit un DTO à la main. C'est l'endpoint typique pour qu'Angular sache « qui est connecté » après un rafraîchissement de page.

L.55-61

`authentifier()` est le vrai cœur du login. `authManager.authenticate(...)` délègue à Spring Security : celui-ci appelle en interne `UserDetailsService.loadUserByUsername` (3.6) puis compare le mot de passe fourni au hash stocké via `PasswordEncoder.matches(...)`. Si ça échoue → `BadCredentialsException`

(interceptée en 3.8, 401). Si ça réussit, on génère le token JWT (3.1) et on construit l' `AuthResponse` renvoyée au client.

BuBackend/.../service/UserService.java

IMPLEMENTS USERDETAILSSERVICE

Le pont entre le modèle métier (`Utilisateur`) et le monde de Spring Security (`UserDetails`).

```
9.  @Service
10.
11.  @Transactional
12.
13.  public class UserService implements UserDetailsService {
14.
15.
16.      private final UtilisateurRepository utilisateurRepository;
17.
18.      private final EmpruntRepository empruntRepository;
19.
20.      private final ReservationRepository reservationRepository;
21.
22.      private final NotificationRepository notificationRepository;
23.
24.      private final PasswordEncoder passwordEncoder;
25.
26.
27.      public UserService(...5 paramètres, un par dépendance ci-dessus...) { /* affectations */ }
28.
29.
30.      @Override
31.
32.      @Transactional(readOnly = true)
33.
34.      public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
35.
36.          Utilisateur user = utilisateurRepository.findByEmail(email)
37.
38.              .orElseThrow(() -> new UsernameNotFoundException("Utilisateur introuvable : " + email));
39.
40.          return User.withUsername(user.getEmail())
41.
42.              .password(user.getMotDePasse())
43.
44.              .roles(user.getRole().name())
45.
46.              .disabled(!user.isActif())
47.
48.              .build();
49.
50.      }
51.
52. }
```

```
3.     public Utilisateur inscrire(String nom, String email, String password) {
4.         if (utilisateurRepository.existsByEmail(email)) {
5.             throw new ConflictException("Email déjà utilisé : " + email);
6.         }
7.         Utilisateur u = new Utilisateur();
8.         u.setNom(nom);
9.         u.setEmail(email);
0.         u.setMotDePasse(passwordEncoder.encode(password));
1.         u.setRole(Utilisateur.Role.ETUDIANT);
2.         return utilisateurRepository.save(u);
3.     }
4.
5.     // createUser(...) : même logique, réservée aux admins (interdit le rôle ADMIN à la création)
6.     // updateUser(...) : met à jour nom/email/mot de passe/rôle/actif, un champ à la fois si non-null
7.
8.     public void deleteUser(Long id) {
9.         Utilisateur u = utilisateurRepository.findById(id)
0.             .orElseThrow(() -> new ResourceNotFoundException("Utilisateur introuvable : " + id));
1.         if (empruntRepository.existsByUtilisateurId(id) || reservationRepository.existsByUtilisateurId(id)) {
2.             throw new ConflictException("Impossible de supprimer un utilisateur ayant un historique d'emprunts "
3.                 + "ou de réservations. Désactivez plutôt son compte.");
4.         }
5.         notificationRepository.deleteByUtilisateurId(id);
6.         utilisateurRepository.delete(u);
7.     }
8.
9.     @Transactional(readOnly = true)
```

```
0.     public Utilisateur parEmail(String email) {  
1.  
2.         return utilisateurRepository.findByEmail(email)  
3.  
4.         .orElseThrow(() -> new UsernameNotFoundException("Utilisateur introuvable : " + email));  
5.  
6.     }  
7.  
8. }
```

L.19-21

`@Service` (spécialisation sémantique de `@Component` pour la couche métier) + `@Transactional` au niveau classe (chaque méthode publique s'exécute dans une transaction par défaut).

`implements UserDetailsService` est l'interface que Spring Security appelle pour charger un utilisateur — c'est ce contrat qui fait que **n'importe quel bean** implémentant cette interface est automatiquement reconnu (voir `SecurityConfig`, qui injecte `UserDetailsService` sans jamais nommer `UserService` explicitement — couplage par interface, pas par implémentation).

L.32-41

`loadByUsername` traduit une entité JPA en objet `UserDetails` générique de Spring Security via le builder `User.withUsername(...)` : email comme identifiant, hash BCrypt tel quel (jamais redécodé), `.roles(...)` (Spring prefixera `ROLE_` automatiquement), `.disabled(!actif)` — un compte désactivé ne peut plus s'authentifier même avec le bon mot de passe.

L.44-52

`inscrire` : vérifie l'unicité applicative de l'email **avant** de toucher la base (évite de laisser remonter une contrainte SQL brute en erreur 500), hache le mot de passe (jamais stocké en clair), force le rôle à `ETUDIANT` — impossible de s'auto-promouvoir bibliothécaire/admin depuis le formulaire d'inscription public.

L.60-67

`deleteUser` refuse la suppression si l'utilisateur a un historique (contrainte métier, pas seulement une FK base) et purge d'abord ses notifications pour éviter une violation de contrainte étrangère résiduelle.

dto/LoginRequest.java • RegisterRequest.java • AuthResponse.java

RECORDS

Les DTOs d'authentification, sous forme de *records* Java (immuables, générés automatiquement : constructeur, accesseurs, `equals` / `hashCode` / `toString`).

```
1. public record LoginRequest(  
2.     @NotBlank(message = "L'email est obligatoire")  
3.     String email,  
4.  
5.     @NotBlank(message = "Le mot de passe est obligatoire")  
6.     String password  
7. ) { }  
8.  
9. public record RegisterRequest(  
0.     @NotBlank(message = "Le nom est obligatoire")  
1.     String nom,  
2.  
3.     @NotBlank(message = "L'email est obligatoire")  
4.     @Email(message = "Email invalide")  
5.     String email,  
6.  
7.     @NotBlank(message = "Le mot de passe est obligatoire")  
8.     @Size(min = 6, message = "Le mot de passe doit contenir au moins 6 caractères")  
9.     String password  
0. ) { }  
1.  
2. public record AuthResponse(  
3.     String token, String type, Long userId, String email, String nom, String role
```


4.) { }

tout le bloc

Pourquoi des DTOs dédiés plutôt que de recevoir directement l'entité `Utilisateur` ? Deux raisons : (1) **sécurité** — un client ne peut pas glisser un champ `role: "ADMIN"` dans le JSON envoyé, puisque `LoginRequest / RegisterRequest` n'ont même pas ce champ (protection contre le *mass assignment*) ; (2) **validation ciblée** — chaque champ porte les annotations Bean Validation adaptées au contrat de l'API, indépendamment des contraintes de la table.

AuthResponse

`type` vaut toujours `"Bearer"` — c'est une convention du standard HTTP
(`Authorization: Bearer <token>`) que le frontend réutilise telle quelle pour construire le header (voir 4.2).

exception/GlobalExceptionHandler.java (+ ErrorResponse, ResourceNotFoundException, ConflictException)

@RESTCONTROLLERADVICE

Le point unique qui traduit chaque exception métier en réponse HTTP cohérente — évite de dupliquer des blocs `try/catch` dans chaque contrôleur.

```
1. package com.projet.bibliotheque.exception;
2.
3. import com.projet.bibliotheque.dto.ErrorResponse;
4. import org.springframework.http.HttpStatus;
5. import org.springframework.http.ResponseEntity;
6. import org.springframework.security.access.AccessDeniedException;
7. import org.springframework.security.authentication.BadCredentialsException;
8. import org.springframework.security.core.userdetails.UsernameNotFoundException;
9. import org.springframework.validation.FieldError;
0. import org.springframework.web.bind.MethodArgumentNotValidException;
1. import org.springframework.web.bind.annotation.ExceptionHandler;
2. import org.springframework.web.bind.annotation.RestControllerAdvice;
3.
4. import java.util.stream.Collectors;
5.
6. @RestControllerAdvice
7. public class GlobalExceptionHandler {
8.
9.     @ExceptionHandler(ResourceNotFoundException.class)
0.     public ResponseEntity<ErrorResponse> notFound(ResourceNotFoundException e) {
1.         return build(HttpStatus.NOT_FOUND, e.getMessage());
2.     }
```

```
3.
4. @ExceptionHandler({ConflictException.class, IllegalStateException.class})
5.     public ResponseEntity<ErrorResponse> conflict(RuntimeException e) {
6.         return build(HttpStatus.CONFLICT, e.getMessage());
7.     }
8.
9. @ExceptionHandler(IllegalArgumentException.class)
10.     public ResponseEntity<ErrorResponse> badRequest(IllegalArgumentException e) {
11.         return build(HttpStatus.BAD_REQUEST, e.getMessage());
12.     }
13.
14. @ExceptionHandler(MethodArgumentNotValidException.class)
15.     public ResponseEntity<ErrorResponse> validation(MethodArgumentNotValidException e) {
16.         String message = e.getBindingResult().getFieldErrors().stream()
17.             .map(FieldError::getDefaultMessage)
18.             .collect(Collectors.joining(" ; "));
19.         return build(HttpStatus.BAD_REQUEST, message.isBlank() ? "Requête invalide" : message);
20.     }
21.
22. @ExceptionHandler({BadCredentialsException.class, UsernameNotFoundException.class})
23.     public ResponseEntity<ErrorResponse> unauthorized(RuntimeException e) {
24.         return build(HttpStatus.UNAUTHORIZED, "Email ou mot de passe incorrect");
25.     }
26.
27. @ExceptionHandler(AccessDeniedException.class)
28.     public ResponseEntity<ErrorResponse> forbidden(AccessDeniedException e) {
29.         return build(HttpStatus.FORBIDDEN, "Accès refusé : privilèges insuffisants");
```

```

0.     }
1.
2.     @ExceptionHandler(Exception.class)
3.     public ResponseEntity<ErrorResponse> general(Exception e) {
4.         return build(HttpStatus.INTERNAL_SERVER_ERROR, "Erreur interne : " + e.getMessage());
5.     }
6.
7.     private ResponseEntity<ErrorResponse> build(HttpStatus status, String message) {
8.         return ResponseEntity.status(status).body(ErrorResponse.of(status.value(), message));
9.     }
0. }

```

L.16

`@RestControllerAdvice` = `@ControllerAdvice` + `@ResponseBody` : une classe auto-détectée par le scan de composants (encore de l'IOC) qui intercepte les exceptions levées par **n'importe quel** `@RestController` de l'application, sans qu'aucun contrôleur n'ait besoin d'un `try/catch`.

L.19-22

Chaque méthode `@ExceptionHandler(X.class)` intercepte le type `X` (ou ses sous-types) levé n'importe où dans la pile d'appel d'une requête, et le traduit en code HTTP précis — ici `ResourceNotFoundException` (voir 3.11, 3.6) → `404`.

L.34-40

`MethodArgumentNotValidException` est levée automatiquement par Spring quand `@Valid` échoue sur un `@RequestBody` (ex: `RegisterRequest` avec un mot de passe de 3 caractères). On agrège tous les messages d'erreur de champ en une seule chaîne lisible plutôt que de renvoyer la structure interne de Spring.

L.42-45

`BadCredentialsException` / `UsernameNotFoundException` → toujours le **même** message générique *"Email ou mot de passe incorrect"*, qu'on ait tapé un mauvais mot de passe ou un email qui n'existe pas — évite de révéler à un attaquant si un email est enregistré.

L.47-50

`AccessDeniedException` → `403`, c'est exactement l'exception que lève `@PreAuthorize` quand le rôle ne correspond pas (voir 3.9), et aussi celle levée manuellement par `CurrentUser.get()` (3.4).

FICHIER	RÔLE
<code>ErrorResponse.java</code>	<i>record</i> <code>(int status, String message, LocalDateTime timestamp)</code> — la forme JSON uniforme de toute erreur renvoyée par l'API, quel que soit le contrôleur.
<code>ResourceNotFoundException.java</code>	<code>RuntimeException</code> minimale, levée quand une entité demandée par id n'existe pas → mappée en 404.
<code>ConflictException.java</code>	<code>RuntimeException</code> minimale, levée pour un conflit d'état métier (doublet, règle violée, stock épuisé) → mappée en 409.

QUESTION PROBABLE DU JURY

« Pourquoi des exceptions personnalisées plutôt que de renvoyer directement un `ResponseEntity` avec le bon code depuis le service ? » — Ça découplerait la couche service (qui ne devrait connaître **aucun** concept HTTP) du protocole de transport. `LivreService` ou `UserService` lèvent des exceptions *métier* (`ConflictException("ISBN déjà existant")`) ; seule la couche web (`GlobalExceptionHandler`) sait qu'un conflit métier devient un `409`. Cette séparation permettrait par exemple de réutiliser ces mêmes services dans un contexte non-HTTP (tâche planifiée, CLI) sans changement.

BuBackend/.../controller/LivreController.java

CRUD REST — REPRÉSENTATIF

Le contrôleur le plus complet pour illustrer les conventions REST du projet ; `AuteurController` , `PenaliteController` , `NotificationController` suivent exactement le même moule (un `@RestController` fin qui délègue tout à un `@Service`).

```
1. package com.projet.bibliotheque.controller;
2.
3. import com.projet.bibliotheque.dto.LivreDto;
4. import com.projet.bibliotheque.dto.LivreRequest;
5. import com.projet.bibliotheque.service.LivreService;
6. import jakarta.validation.Valid;
7. import org.springframework.http.HttpStatus;
8. import org.springframework.http.ResponseEntity;
9. import org.springframework.security.access.prepost.PreAuthorize;
10. import org.springframework.web.bind.annotation.*;
11.
12. import java.util.List;
13.
14. @RestController
15. @RequestMapping("/api/livres")
16. public class LivreController {
17.
18.     private final LivreService livreService;
19.
20.     public LivreController(LivreService livreService) {
21.
22.         this.livreService = livreService;
23.     }
24. }
```

```
3.
4. @GetMapping
5. public ResponseEntity<List<LivreDto>> listerTous() {
6.     return ResponseEntity.ok(livreService.listerTous());
7. }
8.
9. @GetMapping("/disponibles")
0. public ResponseEntity<List<LivreDto>> disponibles() {
1.     return ResponseEntity.ok(livreService.listerDisponibles());
2. }
3.
4. @GetMapping("/recherche")
5. public ResponseEntity<List<LivreDto>> rechercher(@RequestParam String q) {
6.     return ResponseEntity.ok(livreService.rechercher(q));
7. }
8.
9. @GetMapping("/{id}")
0. public ResponseEntity<LivreDto> trouverParId(@PathVariable Long id) {
1.     return ResponseEntity.ok(livreService.trouverParId(id));
2. }
3.
4. @PostMapping
5. @PreAuthorize("hasAnyRole('BIBLIOTHECAIRE', 'ADMIN')")
6. public ResponseEntity<LivreDto> creer(@Valid @RequestBody LivreRequest req) {
7.     return ResponseEntity.status(HttpStatus.CREATED).body(livreService.creer(req));
8. }
9.
0. @PutMapping("/{id}")
```

```

1.     @PreAuthorize("hasAnyRole('BIBLIOTHECAIRE','ADMIN')")
2.
3.     public ResponseEntity<LivreDto> modifier(@PathVariable Long id, @Valid @RequestBody LivreRequest req) {
4.
5.         return ResponseEntity.ok(livreService.modifier(id, req));
6.     }
7.
8.     @DeleteMapping("/{id}")
9.
10.    @PreAuthorize("hasAnyRole('BIBLIOTHECAIRE','ADMIN')")
11.
12.    public ResponseEntity<Void> supprimer(@PathVariable Long id) {
13.
14.        livreService.supprimer(id);
15.
16.        return ResponseEntity.noContent().build();
17.    }
18.
19. }

```

L.24-27

`@GetMapping` sans chemin = `GET /api/livres`. Le type de retour `ResponseEntity<List<LivreDto>>` donne un contrôle explicite sur le code HTTP (ici `200` via `.ok(...)`) en plus du corps JSON — jamais l'entité `Livre` elle-même (voir 3.11, `DtoMapper`).

L.34-37

`@RequestParam String q` lit un paramètre de requête (`?q=...`) — pour une recherche libre, par opposition à `@PathVariable` qui lit un segment d'URL (`/ {id}`) pour identifier une ressource précise.

L.44-47

`@PreAuthorize("hasAnyRole(...)")` ajoute une **deuxième** couche d'autorisation, en plus des règles d'URL de `SecurityConfig` (3.3) — évaluée par AOP juste avant l'exécution de la méthode grâce à `@EnableMethodSecurity` (voir 1.1 pour le point d'activation). `@Valid @RequestBody LivreRequest req` déclenche la validation Bean Validation (voir `LivreRequest`, champs `@NotBlank` / `@Size` / `@Min`) avant que le corps de la méthode ne s'exécute.

L.56-59

`DELETE` réussi → `204 No Content` (`ResponseEntity<Void>`, `.noContent().build()`) : par convention REST, une suppression réussie ne renvoie pas de corps.

Vérbes HTTP utilisés dans ce contrôleur :

GET lecture, sans effet de bord, public pour le catalogue · **POST** création, réservé
bibliothécaire/admin, **201** · **PUT** remplacement complet, **200** · **DELETE** suppression, **204**

BuBackend/.../controller/EmpruntController.java

ACTIONS MÉTIER + PATCH

Complète le tableau des conventions REST : actions imbriquées sur une ressource (`PATCH /id/prolonger`) et endpoints « scopés » à l'utilisateur connecté sans jamais lui faire préciser son propre id.

```
5.  @RestController
6.  @RequestMapping("/api/emprunts")
7.  public class EmpruntController {
8.
9.      private final EmpruntService empruntService;
10.     private final CurrentUser currentUser;
11.
12.     public EmpruntController(EmpruntService empruntService, CurrentUser currentUser) {
13.
14.         this.empruntService = empruntService;
15.         this.currentUser = currentUser;
16.     }
17.
18.     /** Emprunter un livre (au nom de l'utilisateur connecté). */
19.     @PostMapping
20.     public ResponseEntity<EmpruntDto> emprunter(@Valid @RequestBody EmpruntRequest req) {
21.
22.         EmpruntDto dto = empruntService.emprunter(currentUser.id(), req.livreId());
23.
24.         return ResponseEntity.status(HttpStatus.CREATED).body(dto);
25.     }
26.
27.     @GetMapping("/mes-emprunts")
28.     public ResponseEntity<List<EmpruntDto>> mesEmprunts() {
29.
30.         return ResponseEntity.ok(empruntService.mesEmprunts(currentUser.id()));
31.     }
32. }
```

```
7.     }
8.
9.     /** Prolonger son propre emprunt. */
0.     @PatchMapping("/{id}/prolonger")
1.     public ResponseEntity<EmpruntDto> prolonger(@PathVariable Long id) {
2.         return ResponseEntity.ok(empruntService.prolonger(id, currentUser.id()));
3.     }
4.
5.     // — Espace bibliothécaire —————
6.
7.     @GetMapping
8.     @PreAuthorize("hasAnyRole('BIBLIOTHECAIRE','ADMIN')")
9.     public ResponseEntity<List<EmpruntDto>> tous() {
0.         return ResponseEntity.ok(empruntService.tous());
1.     }
2.
3.     @GetMapping("/retards")
4.     @PreAuthorize("hasAnyRole('BIBLIOTHECAIRE','ADMIN')")
5.     public ResponseEntity<List<EmpruntDto>> retards() {
6.         return ResponseEntity.ok(empruntService.retards());
7.     }
8.
9.     /** Enregistrer le retour d'un livre (guichet bibliothécaire). */
0.     @PatchMapping("/{id}/retour")
1.     @PreAuthorize("hasAnyRole('BIBLIOTHECAIRE','ADMIN')")
2.     public ResponseEntity<EmpruntDto> retourner(@PathVariable Long id) {
3.         return ResponseEntity.ok(empruntService.retourner(id));
4.     }
```

5. }

L.29-32

`emprunter` : le corps de la requête (`EmpruntRequest`) ne contient que `livreId` — jamais l'id de l'emprunteur, systématiquement déduit de `currentUser.id()` (3.4). Impossible d'emprunter « pour quelqu'un d'autre » en modifiant le JSON.

L.40, L.60

`@PatchMapping("/{id}/prolonger")` et `/ {id}/retour` : convention REST pour une **action** qui ne correspond ni à un remplacement complet (`PUT`) ni à une simple lecture — un sous-chemin verbal sur la ressource, avec `PATCH` car seul un sous-ensemble de l'état de l'emprunt change (son statut, sa date de retour).

L.35-53

Même contrôleur, deux niveaux de droits : les trois premières méthodes n'ont **aucun** `@PreAuthorize` (accessibles à tout utilisateur authentifié, cf. `anyRequest().authenticated()` en 3.3), les suivantes exigent explicitement `BIBLIOTHECAIRE` ou `ADMIN`.

BuBackend/.../service/LivreService.java + DtoMapper.java

@SERVICE — LOGIQUE MÉTIER

Le contrôleur ne fait que déléguer ; c'est ici que vivent les règles (unicité d'ISBN, gestion du stock, interdiction de suppression) et la conversion entité → DTO.

```
9.  @Service
10.  @Transactional
11.  public class LivreService {
12.
13.      private final LivreRepository livreRepository;
14.      private final AuteurRepository auteurRepository;
15.      private final ReservationRepository reservationRepository;
16.      private final EmpruntRepository empruntRepository;
17.      private final DtoMapper mapper;
18.
19.      // constructeur : 5 dépendances
20.
21.      @Transactional(readonly = true)
22.      public List<LivreDto> listerTous() {
23.
24.          return livreRepository.findAll().stream().map(this::toDto).toList();
25.
26.      }
27.
28.      // listerDisponibles(), trouverParId(), rechercher() : même schéma @Transactional(readonly = true)
29.
30.      public LivreDto creer(LivreRequest req) {
31.
32.          if (req.isbn() != null && !req.isbn().isBlank() && livreRepository.existsByIsbn(req.isbn())) {
33.
34.              throw new ConflictException("ISBN déjà existant : " + req.isbn());
35.
36.          }
37.
38.          Livre livre = new Livre();
```

```
2.     appliquer(livre, req);
3.
4.     return toDto(livreRepository.save(livre));
5.
6. }
7.
8. public void supprimer(Long id) {
9.
10.     Livre livre = getLivre(id);
11.
12.     if (livre.getStockDisponible() < livre.getStockTotal()) {
13.
14.         throw new ConflictException("Impossible de supprimer un livre avec des exemplaires empruntés ou réservés");
15.
16.     }
17.
18.     if (empruntRepository.existsByLivreId(id) || reservationRepository.existsByLivreId(id)) {
19.
20.         throw new ConflictException("Impossible de supprimer un livre ayant un historique d'emprunts "
21.
22.             + "ou de réservations");
23.
24.     }
25.
26.     livreRepository.delete(livre);
27.
28. }
29.
30. private void appliquer(Livre livre, LivreRequest req) {
31.
32.     // copie les champs du DTO vers l'entité...
33.
34.     List<Auteur> auteurs = req.auteurIds().stream()
35.
36.         .map(aid -> auteurRepository.findById(aid)
37.
38.             .orElseThrow(() -> new ResourceNotFoundException("Auteur introuvable : " + aid)))
39.
40.         .toList();
41.
42.     livre.setAuteurs(new java.util.ArrayList<>(auteurs));
43.
44. }
45.
46. private LivreDto toDto(Livre l) {
47.
48.     int fileAttente = (int) reservationRepository.countByLivreIdAndStatut(l.getId(), Reservation.Statut.EN_ATTENTE);
49.
50.     return mapper.toLivreDto(l, fileAttente);
51. }
```

```
0.     }
1. }
```

L.31, L.60-64

`@Transactional(readOnly = true)` sur les lectures : indique à Hibernate qu'aucune écriture n'est attendue (petite optimisation, pas de suivi des modifications), et documente l'intention. Les méthodes d'écriture (`créer`, `supprimer` ...) restent sur le `@Transactional` par défaut de la classe (lecture **et** écriture, commit automatique en fin de méthode si aucune exception, rollback sinon).

L.29-35

`créer` vérifie l'unicité de l'ISBN **avant** d'écrire (409 propre plutôt qu'une violation de contrainte SQL qui remonterait en 500) — le même principe que dans `UserService.inscrire()`.

L.38-46

`supprimer` refuse si des exemplaires sont encore empruntés/réservés, **et** si un historique existe — deux gardes distinctes pour deux raisons différentes (état courant vs intégrité historique), toutes deux en `ConflictException` → 409.

L.55-58

`appliquer()` résout chaque id d'auteur reçu en entité réelle via `auteurRepository.findById(...)`, puis `livre.setAuteurs(...)` — c'est cet appel, côté propriétaire de la relation (voir 2.2), qui réécrit la table de jointure `livre_auteur`.

L.66-67

`toDto()` calcule la file d'attente **à la volée** (elle n'est pas stockée sur l'entité `Livre`) avant de déléguer au `DtoMapper`.

DtoMapper — une seule responsabilité : entité JPA → record DTO, jamais l'inverse.

```
7. public LivreDto toLivreDto(Livre l, int fileAttente) {
8.     return new LivreDto(
9.         l.getId(), l.getTitre(), l.getIsbn(), l.getAnneePublication(), l.getCategorie(),
0.         l.getStockTotal(), l.getStockDisponible(),
1.         l.getStockDisponible() != null && l.getStockDisponible() > 0,
2.         fileAttente,
3.         l.getAuteurs().stream().map(this::toAuteurSimple).toList(),
```

```
4.         l.getImageUrl(), l.getDescription()
5.     );
6. }
```

L.31

`disponible` est un champ **calculé**, absent de l'entité `Livre` : le DTO peut exposer des champs dérivés que le client n'a pas à recalculer lui-même.

L.33

`l.getAuteurs()` est ici appelé **à l'intérieur** de la méthode de service `@Transactional` (via `toDto`) — donc la session Hibernate est encore ouverte et le chargement `LAZY` (2.2) fonctionne. Si ce même appel était fait depuis le contrôleur, il échouerait (voir 2.7, `open-in-view=false`).

NOTION CLÉ

Toutes les méthodes `toXxxDto` de `DtoMapper` (Auteur, Livre, Utilisateur, Emprunt, Reservation, Penalite, Notification) suivent le même principe : elles ne prennent **que** des entités déjà chargées et retournent un *record* immutable. Aucune ne touche un repository — la responsabilité est strictement la conversion.

4 Frontend Angular — client du JWT

Le professeur n'a pas cité Angular dans sa liste, mais c'est ici que se joue le test « supprimer le token »

Angular a son propre conteneur d'injection de dépendances, indépendant de Spring : la fonction `inject()` y joue le même rôle qu'un constructeur Spring — demander un service sans savoir comment il est construit. Ces six fichiers forment la chaîne complète : se connecter, mémoriser le token, l'attacher à chaque appel, protéger les routes.

La source de vérité unique côté client : où vit le token, et qui est connecté.

```
1. import { Injectable, computed, inject, signal } from '@angular/core';
2. import { HttpClient } from '@angular/common/http';
3. import { Observable, tap } from 'rxjs';
4. import { AuthResponse, Role, Utilisateur } from '../models';
5.
6. const TOKEN_KEY = 'biblio_token';
7. const USER_KEY = 'biblio_user';
8.
9. @Injectable({ providedIn: 'root' })
0. export class AuthService {
1.   private http = inject(HttpClient);
2.
3.   // Utilisateur courant exposé en signal (réactif dans tout le layout)
4.   private currentUser = signal<AuthResponse | null>(this.restore());
5.
6.   readonly user = this.currentUser.asReadonly();
7.   readonly isLoggedIn = computed(() => this.currentUser() !== null);
8.   readonly role = computed<Role | null>(() => this.currentUser()?.role ?? null);
9.   readonly isBibliothecaire = computed(() => {
0.     const r = this.currentUser()?.role;
1.     return r === 'BIBLIOTHECAIRE' || r === 'ADMIN';
2.   });
3.   readonly isAdmin = computed(() => this.currentUser()?.role === 'ADMIN');
4.
```

```
5. login(email: string, password: string): Observable<AuthResponse> {
6.     return this.http.post<AuthResponse>('/api/auth/login', { email, password })
7.     .pipe(tap(res => this.persist(res)));
8. }
9.
0. register(nom: string, email: string, password: string): Observable<AuthResponse> {
1.     return this.http.post<AuthResponse>('/api/auth/register', { nom, email, password })
2.     .pipe(tap(res => this.persist(res)));
3. }
4.
5. me(): Observable<Utilisateur> {
6.     return this.http.get<Utilisateur>('/api/auth/me');
7. }
8.
9. /** Déconnexion complète avec suppression des données locales */
0. logout(): void {
1.     localStorage.removeItem(TOKEN_KEY);
2.     localStorage.removeItem(USER_KEY);
3.     this.currentUser.set(null);
4. }
5.
6. /** Nettoie la session sans rediriger (utilisé par la page d'accueil) */
7. clearSession(): void {
8.     localStorage.removeItem(TOKEN_KEY);
9.     localStorage.removeItem(USER_KEY);
0.     this.currentUser.set(null);
1. }
```

```

2.
3.   get token(): string | null {
4.
5.       return localStorage.getItem(TOKEN_KEY);
6.
7.   }
8.
9.   private persist(res: AuthResponse): void {
10.
11.       localStorage.setItem(TOKEN_KEY, res.token);
12.
13.       localStorage.setItem(USER_KEY, JSON.stringify(res));
14.
15.       this.currentUser.set(res);
16.   }
17.
18.   private restore(): AuthResponse | null {
19.
20.       const raw = localStorage.getItem(USER_KEY);
21.
22.       return raw ? JSON.parse(raw) as AuthResponse : null;
23.   }
24. }

```

L.9-10

`@Injectable({ providedIn: 'root' })` est l'équivalent Angular d'un `@Service` Spring : le service devient un singleton disponible dans toute l'application, instancié par l'injecteur racine la première fois qu'un composant le demande.

L.11

`inject(HttpClient)` est la syntaxe fonctionnelle moderne d'Angular (alternative à l'injection par constructeur) : équivalent conceptuel à un `private final X x;` reçu en paramètre de constructeur côté Spring.

L.13

`signal<AuthResponse | null>(this.restore())` : un *signal* Angular (état réactif). Sa valeur initiale est calculée **une seule fois**, à la construction du service — donc au chargement de l'application — en lisant `localStorage` via `restore()`.

L.17

`isLoggedIn` ne regarde **jamais** directement le `localStorage` : c'est une valeur *calculée* (`computed`) à partir du signal en mémoire `currentUser` . C'est le détail technique le plus important pour le test du token (voir section 6).

L.25-27, L.30-32

`login / register` retournent un `Observable<AuthResponse>` `RxJS`.

`.pipe(tap(res => this.persist(res)))` : `tap` exécute un effet de bord (sauvegarder la session) sans transformer la donnée qui continue vers l'abonné (le composant `Login` , voir 4.6).

L.39-43

`logout()` est la **seule** voie « propre » de déconnexion : supprime les deux clés de `localStorage` et remet le signal à `null` — donc `isLoggedIn()` redevient `false` immédiatement (le signal étant en mémoire, tous les composants qui le lisent se remettent à jour automatiquement, sans rechargement de page).

L.53-55

Le *getter* `token` , à l'inverse de `isLoggedIn` , relit `localStorage` **en direct à chaque appel** — jamais mis en cache. C'est ce que l'intercepteur HTTP (4.2) appelle avant chaque requête.

PIÈGE À CONNAÎTRE — LA NUANCE DU TEST « SUPPRIMER LE TOKEN »

`isLoggedIn()` (relit le signal, mis à jour uniquement par `persist()` / `logout()` / `clearSession()` ou au rechargement de la page) et `token` (relit `localStorage` en direct) **peuvent temporairement diverger**. Si on supprime `biblio_token` depuis les DevTools sans recharger la page, `isLoggedIn()` reste `true` jusqu'au prochain appel API — c'est ce dernier qui échouera et déclenchera la vraie déconnexion, via l'intercepteur (4.2). Détail exact dans le test 1 (section 6).

BuFrontend/src/app/core/auth.interceptor.ts

HTTPINTERCEPTORFN

L'équivalent frontend de `JwtAuthFilter` (3.2) : un point de passage unique pour **toutes** les requêtes HTTP sortantes.

```
1. import { HttpResponse, HttpInterceptorFn } from '@angular/common/http';
2. import { inject } from '@angular/core';
3. import { Router } from '@angular/router';
4. import { catchError, throwError } from 'rxjs';
5. import { AuthService } from '../auth.service';
6.
7. /**
8.  * Ajoute le header Bearer sur les requêtes /api (sauf auth) et redirige vers
9.  * /login en cas de 401 (token expiré/invalidé).
10. */
11. export const authInterceptor: HttpInterceptorFn = (req, next) => {
12.     const auth = inject(AuthService);
13.     const router = inject(Router);
14.     const token = auth.token;
15.
16.     const isAuthRoute = req.url.includes('/api/auth/login') || req.url.includes('/api/auth/register');
17.     const authReq = token && !isAuthRoute
18.       ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
19.       : req;
20.
21.     return next(authReq).pipe(
22.       catchError((err: HttpResponse) => {
23.         if (err.status === 401 && !isAuthRoute) {
```

```
4.     auth.logout();
5.     router.navigate(['/login']);
6.   }
7.   return throwError(() => err);
8. })
9. );
10.};
```

L.11

`HttpInterceptorFn` : la forme *fonctionnelle* moderne des intercepteurs Angular (17+), plus légère qu'une classe implémentant `HttpInterceptor`. `inject()` fonctionne même ici, hors constructeur, car Angular exécute la fonction dans un contexte d'injection valide.

L.16

`isAuthRoute` exclut `/login` et `/register` : on n'attache jamais un vieux token à une tentative de connexion, et une 401 sur ces routes ne doit **pas** déclencher la logique de déconnexion (ce serait absurde — on n'est pas encore connecté).

L.17-19

`req.clone({ setHeaders: { ... } })` : les requêtes Angular (`HttpRequest`) sont **immuables** — on ne les modifie jamais en place, on en clone une nouvelle avec le header `Authorization: Bearer <token>` ajouté. Si `token` est `null` (déconnecté), la requête `req` originale part sans ce header.

L.23-29

Le trajet **retour** : `catchError` intercepte toute erreur HTTP. `err.status === 401` signifie précisément « le serveur ne reconnaît pas ce token comme valide » (voir `authenticationEntryPoint`, 3.3) → on appelle `auth.logout()` (efface tout, y compris le signal `isLoggedIn`) puis on redirige vers `/login`. C'est le mécanisme réel qui répond à « que se passe-t-il si je supprime/invalid le token ».

La barrière **avant** même que la page ne s'affiche — contrairement à l'intercepteur qui réagit après une requête déjà partie.

```
1. import { inject } from '@angular/core';
2. import { CanActivateFn, Router } from '@angular/router';
3. import { AuthService } from '../auth.service';
4.
5. /** Exige une session active, sinon redirige vers /login avec returnUrl. */
6. export const authGuard: CanActivateFn = (route, state) => {
7.     const auth = inject(AuthService);
8.     const router = inject(Router);
9.
10.    if (auth.isLoggedIn()) {
11.        return true;
12.    }
13.
14.    router.navigate(['/login'], { queryParams: { returnUrl: state.url } });
15.
16.    return false;
17. };
18.
19. /** Réserve aux bibliothécaires / admins. */
20. export const bibliothecaireGuard: CanActivateFn = (route, state) => {
21.     const auth = inject(AuthService);
22.     const router = inject(Router);
23.
24.     if (auth.isBibliothecaire()) {
```



```
4.     return true;

5.   }

6.

7.   router.navigate(['/login'], { queryParams: { returnUrl: state.url } });

8.   return false;

9. };

0.

1.  /** Réserve aux administrateurs uniquement. */

2.  export const adminGuard: CanActivateFn = (route, state) => {

3.    const auth = inject(AuthService);

4.    const router = inject(Router);

5.

6.    if (auth.isAdmin()) {

7.      return true;

8.    }

9.

0.    router.navigate(['/login'], { queryParams: { returnUrl: state.url } });

1.    return false;

2.  };
```

L.6

`CanActivateFn` : signature de garde fonctionnelle appelée par le routeur Angular **avant** d'activer une route. Si elle renvoie `false` (ou redirige), la navigation est annulée.

L.10-12

Trois gardes, trois niveaux, même schéma : `authGuard` vérifie juste une session (`isLoggedIn()`), `bibliothecaireGuard` / `adminGuard` vérifient en plus le rôle (`isBibliothecaire()` / `isAdmin()` — des `computed` définis dans `AuthService` , 4.1).

L.14

`{ queryParams: { returnUrl: state.url } }` : mémorise l'URL initialement demandée dans l'URL de redirection, pour pouvoir y renvoyer l'utilisateur juste après une connexion réussie.

QUESTION PROBABLE DU JURY

« Ces gardes suffisent-elles à sécuriser l'application ? » — Non, et c'est un point important à assumer : les gardes Angular ne sont qu'un **confort d'UX** (cacher un lien, éviter un aller-retour réseau inutile). La vraie sécurité est **toujours** côté serveur (`SecurityConfig` + `@PreAuthorize` , 3.3/3.9) — un utilisateur techniquement averti pourrait appeler l'API directement en contournant totalement Angular ; c'est le serveur qui doit refuser, jamais seulement le client.

BuFrontend/src/app/app.routes.ts

ROUTES PROTÉGÉES

```
1. import { Routes } from '@angular/router';
2. import { adminGuard, authGuard, bibliothecaireGuard } from '../core/guards';
3.
4. export const routes: Routes = [
5.   { path: '', pathMatch: 'full', redirectTo: 'accueil' },
6.
7.   { path: 'login', loadComponent: () => import('../pages/login/login').then(m => m.Login) },
8.   { path: 'register', loadComponent: () => import('../pages/register/register').then(m => m.Register) },
9.   { path: 'accueil', loadComponent: () => import('../pages/accueil/accueil').then(m => m.Accueil) },
10.
11.   { path: 'catalogue', canActivate: [authGuard], loadComponent: () => import('../pages/catalogue/catalogue').then(m => m.Cat
12.   { path: 'catalogue/:id', canActivate: [authGuard], loadComponent: () => import('../pages/livre-detail/livre-detail').then
13.
14.   // Espace étudiant (authentifié)
15.
16.   { path: 'mes-emprunts', canActivate: [authGuard], loadComponent: () => import(...).then(m => m.MesEmprunts) },
17.   { path: 'mes-reservations', canActivate: [authGuard], loadComponent: () => import(...).then(m => m.MesReservations) },
18.   { path: 'mes-penalites', canActivate: [authGuard], loadComponent: () => import(...).then(m => m.MesPenalites) },
19.
20.   // Espace bibliothécaire / admin
21.   { path: 'dashboard', canActivate: [bibliothecaireGuard], loadComponent: () => import(...).then(m => m.DashboardPage) },
22.   { path: 'gestion-livres', canActivate: [bibliothecaireGuard], loadComponent: () => import(...).then(m => m.GestionLivres
23.   { path: 'gestion-auteurs', canActivate: [bibliothecaireGuard], loadComponent: () => import(...).then(m => m.GestionAuteu
24.   { path: 'gestion-emprunts', canActivate: [bibliothecaireGuard], loadComponent: () => import(...).then(m => m.GestionEmpr
25.   { path: 'gestion-penalites', canActivate: [bibliothecaireGuard], loadComponent: () => import(...).then(m => m.GestionPen
26.   { path: 'gestion-utilisateurs', canActivate: [adminGuard], loadComponent: () => import(...).then(m => m.GestionUtilisate
```

```
6.  
7.    // Redirection par défaut vers Le catalogue (qui sera protégé)  
8.    { path: '**', redirectTo: 'catalogue' },  
9.  ];
```

L.7-9

`loadComponent: () => import(...).then(...)` : chargement **paresseux** (lazy loading) — le code de chaque page n'est téléchargé par le navigateur qu'au moment où l'utilisateur navigue vers cette route, pas au démarrage de l'application.

L.11-25

`canActivate: [authGuard]` (ou `bibliothecaireGuard` / `adminGuard`) place la garde correspondante (4.3) devant chaque route sensible — le même tableau des trois niveaux de droits que côté backend (3.3), mais appliqué aux *pages* plutôt qu'aux *endpoints*.

L.28

`{ path: '**', redirectTo: 'catalogue' }` : route « attrape-tout » — toute URL inconnue redirige vers le catalogue (qui redirigera lui-même vers `/login` si non connecté, via `authGuard`).

L'équivalent Angular de `BeansConfig` + `SecurityConfig` réunis : où l'on déclare, une fois pour toute l'application, quels services/mécanismes sont disponibles.

```
1. import { ApplicationConfig, provideBrowserGlobalErrorListeners, provideZonelessChangeDetection } from '@angular/core';
2. import { provideRouter } from '@angular/router';
3. import { provideHttpClient, withInterceptors } from '@angular/common/http';
4. import { provideAnimationsAsync } from '@angular/platform-browser/animations/async';
5.
6. import { routes } from './app.routes';
7. import { authInterceptor } from './core/auth.interceptor';
8.
9. export const appConfig: ApplicationConfig = {
10.   providers: [
11.     provideBrowserGlobalErrorListeners(),
12.     provideZonelessChangeDetection(),
13.     provideRouter(routes),
14.     provideHttpClient(withInterceptors([authInterceptor])),
15.     provideAnimationsAsync(),
16.   ],
17. };;
```

L.9-16

Composants Angular « standalone » (pas de `NgModule`) : tout le câblage de l'application se fait ici, dans un simple tableau `providers` — chaque fonction `provideXxx()` enregistre un ensemble de services dans l'injecteur racine.

L.14

`provideHttpClient(withInterceptors([authInterceptor]))` : c'est **cette ligne précise** qui branche `authInterceptor` (4.2) sur le pipeline HTTP global — sans elle, aucune requête n'aurait jamais le header `Authorization` ajouté automatiquement.

BuFrontend/src/app/pages/login/login.ts

COMPOSANT

```
2. export class Login {
3.     private fb = inject(FormBuilder);
4.     private auth = inject(AuthService);
5.     private router = inject(Router);
6.     private ui = inject(Ui);
7.
8.     loading = signal(false);
9.
10.    form = this.fb.nonNullable.group({
11.        email: ['', [Validators.required, Validators.email]],
12.        password: ['', [Validators.required]],
13.    });
14.
15.    // comptesDemo : Les 3 comptes de démonstration pré-remplis (voir section 5)
16.
17.    submit(): void {
18.        if (this.form.invalid) return;
19.        this.loading.set(true);
20.        const { email, password } = this.form.getRawValue();
21.        this.auth.login(email, password).subscribe({
22.            next: () => {
23.                this.ui.success('Connexion réussie');
24.                this.router.navigate(['/catalogue']);
25.            },
26.            error: err => {
```

```
7.         this.loading.set(false);  
8.  
9.         this.ui.error(err);  
10.    },  
11.    });  
12.  }  
13. }
```

L.23-26

Quatre services injectés par `inject()` : `FormBuilder` (Angular Forms), `AuthService` (4.1), `Router`, et `Ui` (un service maison pour les notifications). Même pattern que partout ailleurs — jamais de `new`.

L.36-38

`auth.login(email, password)` renvoie l'`Observable` vu en 4.1 ; `.subscribe({...})` déclenche réellement l'appel HTTP (un `Observable` RxJS ne fait rien tant que personne ne s'y abonne).

L.37-40

`next` ne s'exécute qu'après que `persist()` (dans `AuthService`, via le `tap` du `.pipe`) a déjà écrit le token en `localStorage` — l'ordre est garanti par RxJS, donc la redirection vers `/catalogue` arrive avec une session déjà valide.

5 Configuration

Deux profils : dev (H2) et par défaut (MySQL, celui du TP)

BuBackend/src/main/resources/application.properties

PROFIL PAR DÉFAUT (MYSQL)

```
1. # — Application —————
2. spring.application.name=bibliotheque
3. server.port=8081
4.
5. # — Base de données (MySQL par défaut, cf. profil "dev" pour H2) —
6. spring.datasource.url=jdbc:mysql://localhost:3306/bibliotheque_db?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=
7. spring.datasource.username=biblio_user
8. spring.datasource.password=adriano
9.
10. # — JPA —————
11. spring.jpa.hibernate.ddl-auto=update
12. spring.jpa.show-sql=false
13. spring.jpa.properties.hibernate.format_sql=true
14. spring.jpa.open-in-view=false
15.
16. # — Taille max des requêtes (upload image en base64 sur /api/livres) —
17. server.tomcat.max-http-form-post-size=5MB
18. server.tomcat.max-swallow-size=5MB
19. spring.servlet.multipart.max-file-size=5MB
20. spring.servlet.multipart.max-request-size=5MB
21.
22. # — JWT —————
```

```

3.  jwt.secret=VotreCleSecreteTresLonguePourJWT2026BibliothequeUniversitaireSpringAngular
4.  jwt.expiration=86400000
5.
6.  # — Règles métier de la bibliothèque —————
7.  biblio.emprunt.duree-jours=14
8.  biblio.emprunt.prolongation-jours=7
9.  biblio.emprunt.max-simultanes=3
0.  biblio.penalite.tarif-journalier=100
1.  biblio.reservation.expiration-heures=48

```

L.3

Port réel du backend : **8081** (et non 8080). C'est cette valeur, héritée par tous les profils sauf s'ils la redéfinissent, qui doit correspondre au `target` du proxy Angular (voir `proxy.conf.json` ci-dessous).

L.22-23

Ce sont exactement les deux propriétés lues par `@Value("${jwt.secret}")` et `@Value("${jwt.expiration}")` dans `JwtService` (3.1). `86400000` ms = 24 heures. Ces clés n'ont **aucun préfixe standard Spring** — elles n'existent que parce que `JwtService` les référence explicitement.

L.26-30

Ce sont les propriétés lues par `BiblioProperties` (1.5) grâce au préfixe `biblio` et à la conversion `kebab-case` → `camelCase` (`duree-jours` → `dureeJours`).

PIÈGE RELEVÉ DANS LE CODE — PORT H2 CONSOLE

Le commentaire de `application-dev.properties` (ci-dessous, L.11) annonce la console H2 sur `http://localhost:8080/h2-console`. C'est faux : le profil `dev` n'y redéfinit pas `server.port`, donc il hérite du `8081` ci-dessus. L'URL réelle est `http://localhost:8081/h2-console` — à vérifier avant une démo live pour ne pas chercher la console au mauvais endroit devant le jury.

BuBackend/src/main/resources/application-dev.properties

PROFIL "DEV" (H2)

```

1.  # Profil DEV : H2 en mémoire, aucune installation requise.
2.  # Lancement : mvn spring-boot:run "-Dspring-boot.run.profiles=dev"

```

```
3. spring.datasource.url=jdbc:h2:mem:bibliotheque_db;DB_CLOSE_DELAY=-1
4.
5. spring.datasource.username=sa
6. spring.datasource.password=
7.
8. spring.jpa.hibernate.ddl-auto=create-drop
9.
10. spring.jpa.show-sql=true
11.
12. # Console H2 : http://localhost:8080/h2-console
13.
14. spring.h2.console.enabled=true
15.
16. spring.h2.console.path=/h2-console
```

L.3

`jdbc:h2:mem:...;DB_CLOSE_DELAY=-1` : base **en mémoire**, détruite à l'arrêt du processus —
`DB_CLOSE_DELAY=-1` évite qu'elle se ferme dès que la première connexion se libère.

L.8

`create-drop` confirmé : le schéma est recréé de zéro à chaque démarrage, ce qui explique pourquoi `DataInitializer` (1.4) réinsère systématiquement les données de démo en profil dev.

Correspondance côté Angular — `BuFrontend/proxy.conf.json`, utilisé par `npm start` pour rediriger les appels `/api/**` vers le backend sans souci de CORS en développement :

```
1. {
2.   "/api": {
3.     "target": "http://localhost:8081",
4.     "secure": false,
5.     "changeOrigin": true
6.   }
7. }
```


6 Tests à exécuter en live

Scénarios concrets, avec le mécanisme exact derrière chaque résultat

Dix scénarios, du plus demandé (« supprimez le token ») au plus révélateur de maîtrise (falsifier le payload). Chacun cite le fichier:ligne responsable du comportement observé — de quoi répondre immédiatement si le jury demande « pourquoi ce code-là et pas une erreur 500 ? ».

01 Supprimer le token → redirection vers /login

Procédure recommandée (fiable, à faire en premier) :

1. Se connecter (ex: compte étudiant), naviguer vers `/catalogue`.
2. Ouvrir les DevTools → onglet Application (Chrome) / Stockage (Firefox) → Local Storage → `http://localhost:4200`.
3. Supprimer les deux clés `biblio_token` **et** `biblio_user` (ou « Clear All »).
4. Rafraîchir la page (F5) sur une route protégée.

RÉSULTAT Redirection immédiate vers `/login`.

Mécanisme : au rechargement, Angular redémarre entièrement → `AuthService` est reconstruit → `restore()` (`auth.service.ts:63-66`) ne trouve plus rien dans `biblio_user` → le signal `currentUser` vaut `null` → `authGuard` (`guards.ts:6-16`) voit `isLoggedIn()` à `false` et redirige.

Variante plus fine (sans recharger la page) — pour montrer que vous comprenez les deux mécanismes :

1. Toujours connecté, sans recharger : supprimer **seulement** `biblio_token` (laisser `biblio_user`).
2. Cliquer sur une action qui déclenche un appel API (changer de page gérée, rafraîchir une liste).

RÉSULTAT La page ne change pas immédiatement au clic ; c'est l'appel API qui échoue, puis la redirection se déclenche.

Mécanisme : `isLoggedIn()` reste `true` (le signal en mémoire n'a pas changé, seul le `localStorage` a bougé) donc le garde de route **laisse passer**. Mais `auth.token` (`auth.service.ts:53-55`) relit `localStorage` en direct et renvoie `null` → la requête part **sans** header `Authorization` (`auth.interceptor.ts:17-19`) → le backend répond `401` (`SecurityConfig.java:75-77`) → l'intercepteur intercepte l'erreur, appelle `auth.logout()` et redirige (`auth.interceptor.ts:24-26`).

CE QUE ÇA PROUVE DEVANT LE JURY

Il existe **deux** lignes de défense indépendantes : le garde de route (confort, immédiat, mais basé sur un état mis en cache) et l'intercepteur+backend (source de vérité réelle, à chaque requête). Un token supprimé finit toujours par être détecté — au pire au prochain appel réseau.

02 Falsifier le token (modifier le rôle dans le payload)

1. Copier la valeur de `biblio_token` depuis le Local Storage.
2. La coller sur jwt.io (ou décoder manuellement le 2^e segment en Base64) pour lire le payload :
`{"sub":"etudiant@universite.sn","role":"ETUDIANT",...}`.
3. Modifier `"role":"ETUDIANT"` en `"role":"ADMIN"`, ré-encoder, recomposer un token avec ce nouveau payload (même header, signature laissée telle quelle ou recalculée sans la vraie clé).
4. Remplacer `biblio_token` par ce token modifié et refaire un appel.

RÉSULTAT 401 — le rôle modifié n'a servi à rien.

Mécanisme : la signature JWT couvre header + payload. Modifier un seul caractère du payload change le résultat attendu de la vérification HMAC ; `Jwts.parser().verifyWith(cle())...parseSignedClaims(token)` (`JwtService.java:53-57`) lève une `JwtException`, interceptée dans `JwtAuthFilter` (`JwtAuthFilter.java:48-51`) qui vide le contexte de sécurité → requête traitée comme non authentifiée → 401.

CE QUE ÇA PROUVE DEVANT LE JURY

Le JWT est *signé*, pas *chiffré* : n'importe qui peut **lire** le rôle, mais personne sans le secret serveur ne peut le **modifier** sans invalider la signature. C'est la garantie d'intégrité, pas de confidentialité.

03 Étudiant qui tente une action bibliothécaire

1. Se connecter avec le compte étudiant (`etudiant@universite.sn`).
2. Depuis un client REST (Postman/Insomnia/curl) ou la console du navigateur, appeler `POST /api/livres` avec le token étudiant.

RÉSULTAT 403 Forbidden — pas 401.

Mécanisme : le token est parfaitement valide et authentifié (contrairement au test 1/2), mais le rôle `ETUDIANT` ne satisfait pas `hasAnyRole('BIBLIOTHECAIRE', 'ADMIN')` (`SecurityConfig.java:60-61` et `LivreController.java:45`) → `accessDeniedHandler` renvoie 403 (`SecurityConfig.java:79-81`).

04 Aucun token du tout

1. Appel direct (curl/Postman) sur une route protégée, sans header `Authorization` :

```
curl http://localhost:8081/api/emprunts/mes-emprunts
```

RÉSULTAT 401 — `{"status":401,"message":"Authentification requise",...}` .

Mécanisme : `anyRequest().authenticated()` (`SecurityConfig.java:70`) rejette toute requête sans authentification ; `authenticationEntryPoint` écrit directement la réponse JSON via `ecrireErreur()` (`SecurityConfig.java:73-77, 87-92`).

05 Inscription avec un email déjà utilisé

1. POST `/api/auth/register` avec `email: "etudiant@universite.sn"` (déjà seedé par `DataInitializer`).

RÉSULTAT 409 Conflict — `"Email déjà utilisé : etudiant@universite.sn"` .

Mécanisme : `UserService.inscrire()` vérifie `existsByEmail(email)` avant d'écrire et lève `ConflictException` (`UserService.java:52-54`), interceptée par `GlobalExceptionHandler.conflict()` (`GlobalExceptionHandler.java:24-27`).

06 Mot de passe trop court à l'inscription

1. POST `/api/auth/register` avec `password: "abc"` (moins de 6 caractères).

RÉSULTAT 400 Bad Request — `"Le mot de passe doit contenir au moins 6 caractères"` .

Mécanisme : `@Size(min = 6)` sur `RegisterRequest.password` (`RegisterRequest.java:16`) + `@Valid` sur le contrôleur (`AuthController.java:37`) déclenchent `MethodArgumentNotValidException`, transformée en 400 par `GlobalExceptionHandler.validation()` (`GlobalExceptionHandler.java:34-40`).

07 Supprimer un livre encore emprunté

1. Se connecter en bibliothécaire.
2. `DELETE /api/livres/{id}` sur « Clean Code » (un exemplaire emprunté par défaut dans les données de démo).

RÉSULTAT 409 Conflict — "Impossible de supprimer un livre avec des exemplaires empruntés ou réservés" .

Mécanisme : `LivreService.supprimer()` compare `stockDisponible` à `stockTotal` avant toute suppression (`LivreService.java:83-86`) — une règle métier applicative, pas une simple contrainte de clé étrangère SQL qui remonterait en 500.

08 Preuve que le mot de passe n'est jamais en clair

1. Lancer le backend en profil `dev` (`SPRING_PROFILES_ACTIVE=dev`).
2. Ouvrir `http://localhost:8081/h2-console` (JDBC URL `jdbc:h2:mem:bibliotheque_db`, user `sa`, mot de passe vide).
3. Exécuter `SELECT email, mot_de_passe FROM utilisateurs;`

RÉSULTAT La colonne affiche `$2a$10$...` (hash BCrypt), jamais `Admin2024!` en clair.

Mécanisme : chaque création d'utilisateur passe par `passwordEncoder.encode(...)` (BCrypt, `BeansConfig.java:19-21`) avant `save()` — voir `DataInitializer.java:122` et `UserService.java:57`. Excellent test visuel à montrer en direct.

09 L'API fonctionne sans Angular

Pour prouver que le contrat réel est REST+JWT, indépendant du frontend :

```
1. curl -X POST http://localhost:8081/api/auth/login \  
2.     -H "Content-Type: application/json" \  
3.     -d "{\"email\":\"admin@universite.sn\",\"password\":\"Admin2024!\"}" \  
4. \  
5. # → { "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5IiwiaWF0IjoxNjU0OTU0MDAwLCJ1aWQiOiJ1", ... } \  
6. \  
7. curl http://localhost:8081/api/auth/me \  
8.     -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5IiwiaWF0IjoxNjU0OTU0MDAwLCJ1aWQiOiJ1"
```

RÉSULTAT Réponses JSON identiques à ce que consomme Angular — aucune dépendance cachée au frontend.

10 (Avancé) Observer une vraie expiration de token

La durée par défaut (`jwt.expiration=86400000` , 24h) est trop longue pour une démo live.

1. Avant la présentation : dans `application.properties` , remplacer temporairement `jwt.expiration=86400000` par `jwt.expiration=20000` (20 secondes) et redémarrer.
2. Se connecter, attendre 20 secondes, refaire un appel API.

RÉSULTAT 401, sans avoir touché au localStorage — le token « naturellement » expiré est rejeté.

Mécanisme : `claims.getExpiration().after(new Date())` devient `false` (`JwtService.java:45-46`). **Ne pas oublier de remettre `86400000` après la démo.**

7 Questions probables & réponses courtes

33 questions, groupées par thème — à
parcourir la veille

IoC & Injection de dépendances

- Q – Qu'est-ce que l'inversion de contrôle ? Donnez un exemple précis dans le code.
- Q – Différence entre @Component, @Service, @Repository, @RestController et @Configuration ?
- Q – Pourquoi le projet n'utilise-t-il jamais @Autowired ?
- Q – Qu'est-ce qu'un bean Spring, et quelle est sa portée par défaut ?
- Q – Pourquoi BeansConfig est-il séparé de SecurityConfig ?
- Q – SecurityConfig appelle jwtAuthFilter() comme une méthode Java normale. Ça ne crée pas un nouvel objet à chaque fois ?
- Q – Pourquoi @ConfigurationProperties plutôt que plusieurs @Value ?
- Q – Comment DataInitializer s'exécute-t-il, et pourquoi ne duplique-t-il pas les données ?
- Q – Qu'est-ce qui déclenche l'auto-configuration de Spring Boot (JPA, sécurité, MVC) ?

JPA & Persistance

- Q – Expliquez la relation Livre ↔ Auteur (N,N). Qui est propriétaire ?
- Q – Pourquoi FetchType.LAZY partout, et quel est le rapport avec open-in-view=false ?
- Q – Comment une méthode comme findByEmail fonctionne-t-elle sans implémentation écrite ?
- Q – Expliquez la requête JPQL rechercher() dans LivreRepository.

Q – Que retourne `List<Object[]>` dans `topLivresEmpruntes()`, et pourquoi pas une entité ?

Q – Quelle stratégie de génération d'identifiant est utilisée ?

Q – Différence entre `ddl-auto=update` (prod) et `create-drop` (dev) ?

Q – Pourquoi ne renvoyez-vous jamais une entité JPA directement dans une réponse REST ?

Q – Que fait `@Transactional`, et pourquoi certaines méthodes ont-elles `readOnly=true` ?

API REST & JWT

Q – Expliquez le trajet complet d'une connexion (login) jusqu'à l'obtention du token.

Q – Expliquez le trajet d'une requête authentifiée jusqu'au contrôleur.

Q – Le contenu du token est-il chiffré ?

Q – Pourquoi `SessionCreationPolicy.STATELESS` ?

Q – Pourquoi le CSRF est-il désactivé ?

Q – Différence entre 401 et 403 dans ce projet, et comment est-ce programmé ?

Q – Comment fonctionne `@PreAuthorize`, et en quoi diffère-t-il des règles de `SecurityConfig` ?

Q – Si je modifie le rôle dans le payload du token et que je le renvoie, ça marche ?

Q – Pourquoi `JwtAuthFilter` est-il placé avant `UsernamePasswordAuthenticationFilter` ?

Q – Pourquoi utiliser des DTOs de validation (`LoginRequest...`) plutôt que l'entité `Utilisateur` ?

Q – N'est-ce pas risqué de stocker le token dans le `localStorage` ?

Frontend Angular

Q – Comment Angular ajoute-t-il le header Authorization à chaque requête ?

Q – Que se passe-t-il si le token est supprimé du navigateur ?

Q – Quel est l'équivalent Angular de l'IoC/DI de Spring ?

Q – Comment les routes protégées empêchent-elles un utilisateur non connecté d'y accéder ?

Comptes de démonstration (seedés par `DataInitializer`, voir 1.4) :

`admin@universite.sn / Admin2024!` · `biblio@universite.sn / Biblio2024!` ·
`etudiant@universite.sn / Etudiant2024!` .

Lancer le projet — Backend dev (H2, sans installation) : `SPRING_PROFILES_ACTIVE=dev`

puis `mvnw -o spring-boot:run` (port 8081). Backend prod (MySQL, celui du TP) : profil par défaut, base `bibliotheque_db`. Frontend : `cd BuFrontend && npm start` (proxy vers `localhost:8081`, voir 5).

Fiche construite directement à partir du code source du projet — chaque numéro de ligne cité correspond au fichier réel au moment de la rédaction.